



US 20250284531A1

(19) **United States**

(12) **Patent Application Publication**

(10) **Pub. No.: US 2025/0284531 A1**

(43) **Pub. Date: Sep. 11, 2025**

Kass et al.

(54) **DYNAMIC SHARING OF GRAPHICS PROCESSING UNIT (GPU) COMPUTATIONAL CAPABILITIES BASED ON PROCESSING DENSITY**

(52) **U.S. Cl.**
CPC **G06F 9/485** (2013.01); **G06T 15/005** (2013.01)

(71) Applicant: **Nvidia Corporation**, Santa Clara, CA (US)

(72) Inventors: **Michael Kass**, Los Altos Hills, CA (US); **Tae Kim**, Los Altos, CA (US); **Georgi M. Chalakov**, Duvall, WA (US)

(21) Appl. No.: **18/597,433**

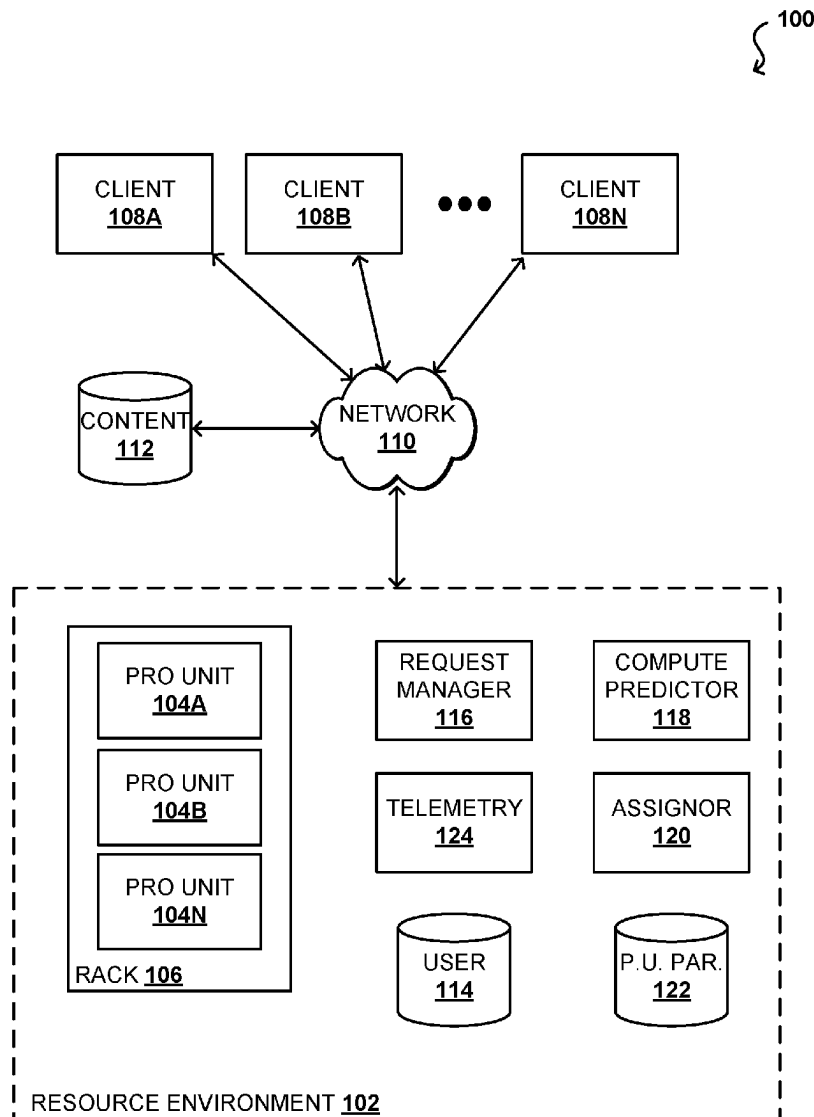
(22) Filed: **Mar. 6, 2024**

Publication Classification

(51) **Int. Cl.**
G06F 9/48 (2006.01)
G06T 15/00 (2011.01)

(57) **ABSTRACT**

Approaches presented herein provide systems and methods for dynamic allocation of processing units to increase computational density. Idle times between sequential processing tasks may be computed and, if the idle time exceeds a threshold capacity, additional sequential processing tasks may be allocated to a common processing unit. As a request, a first portion of a first sequential processing task may be executed, then a second portion of a second sequential processing task may be executed prior to executing a subsequent portion of the second sequential processing task. By using the idle time between portions of sequential processing tasks, output perform may be maintained while using additional processing capabilities that would otherwise remain idle.



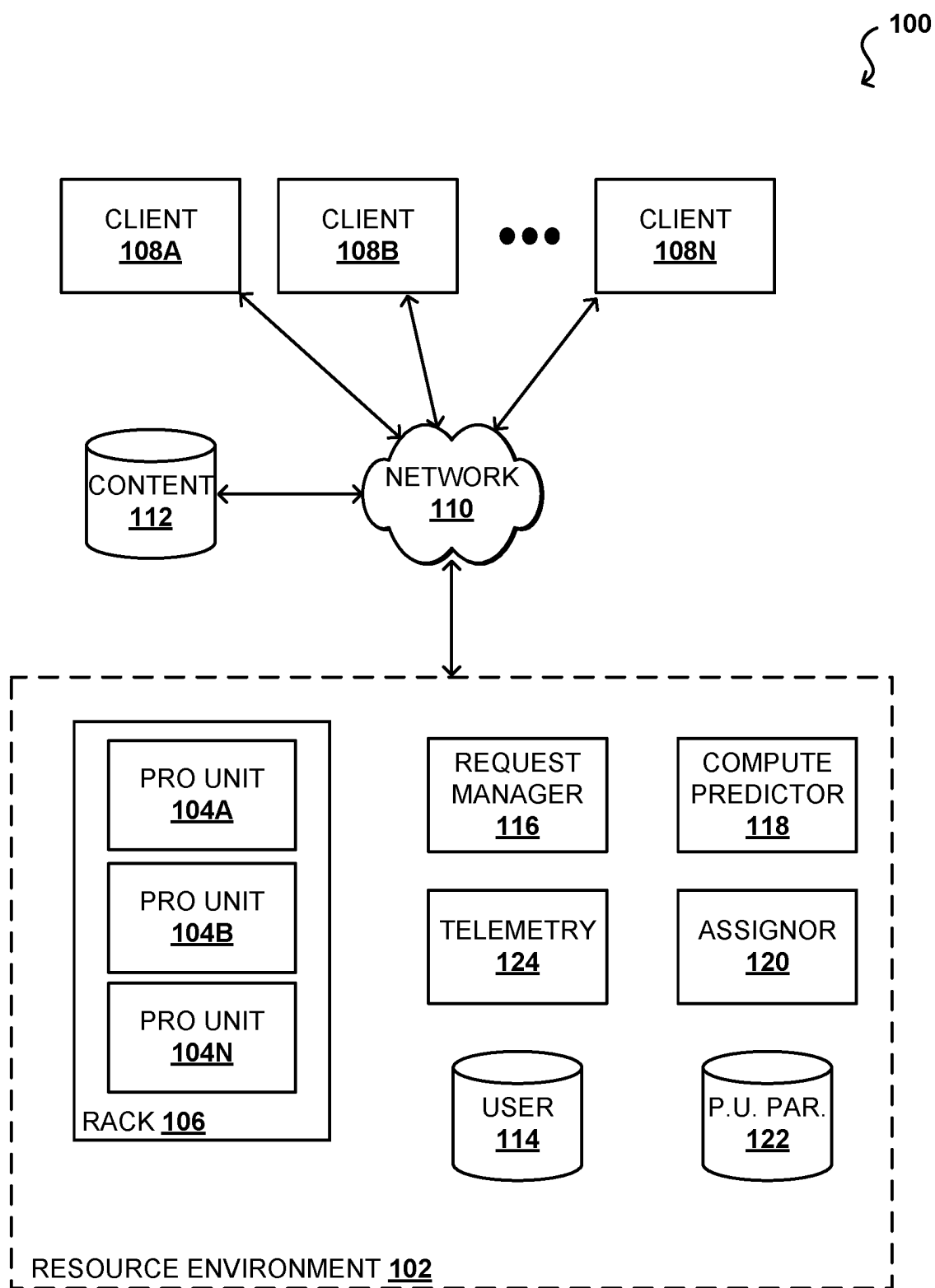


FIG. 1

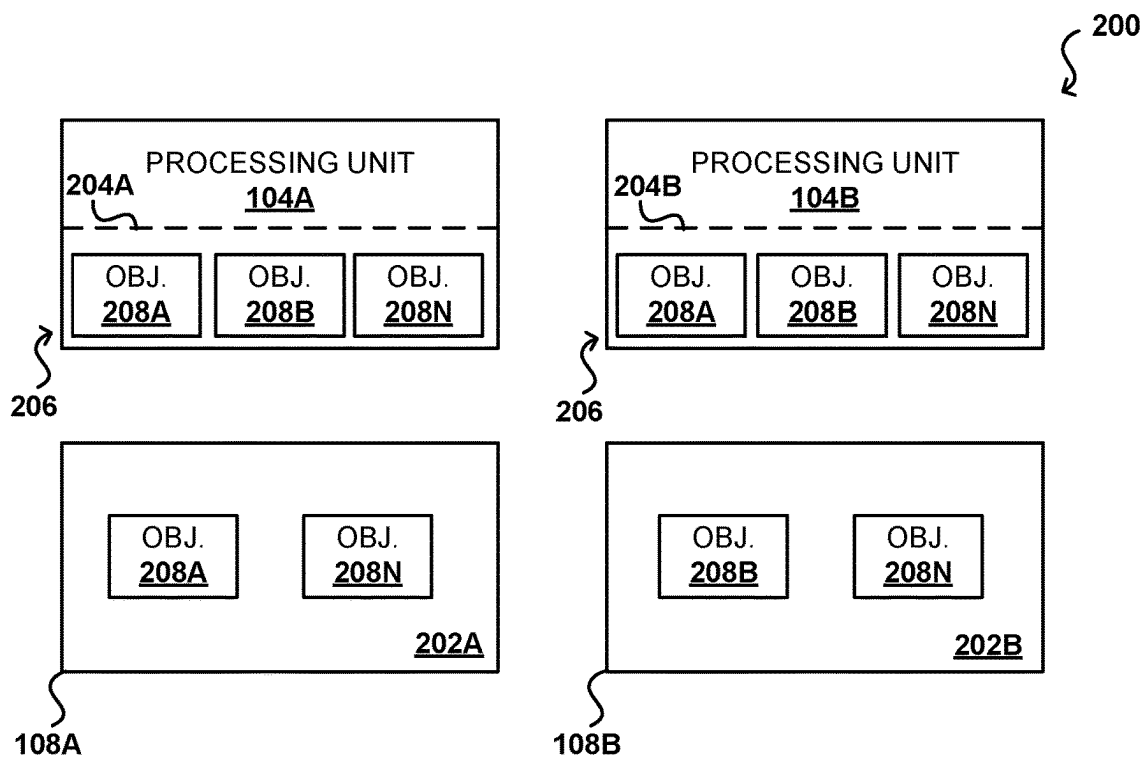


FIG. 2A

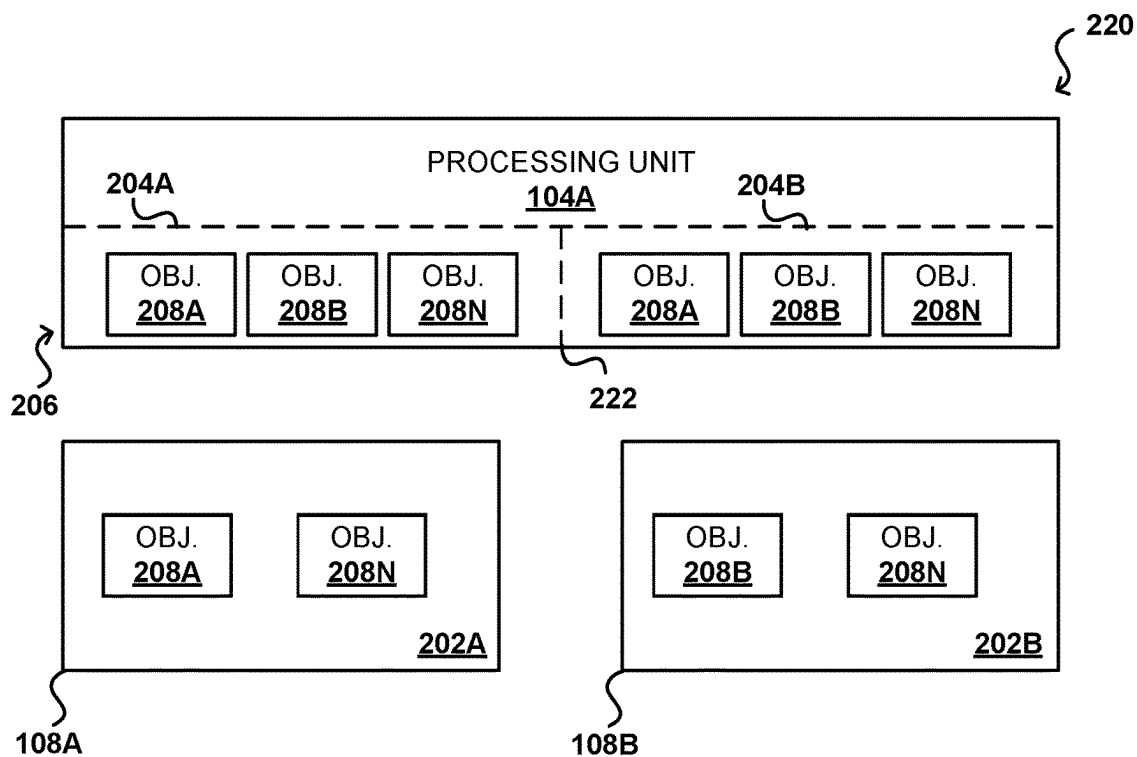


FIG. 2B

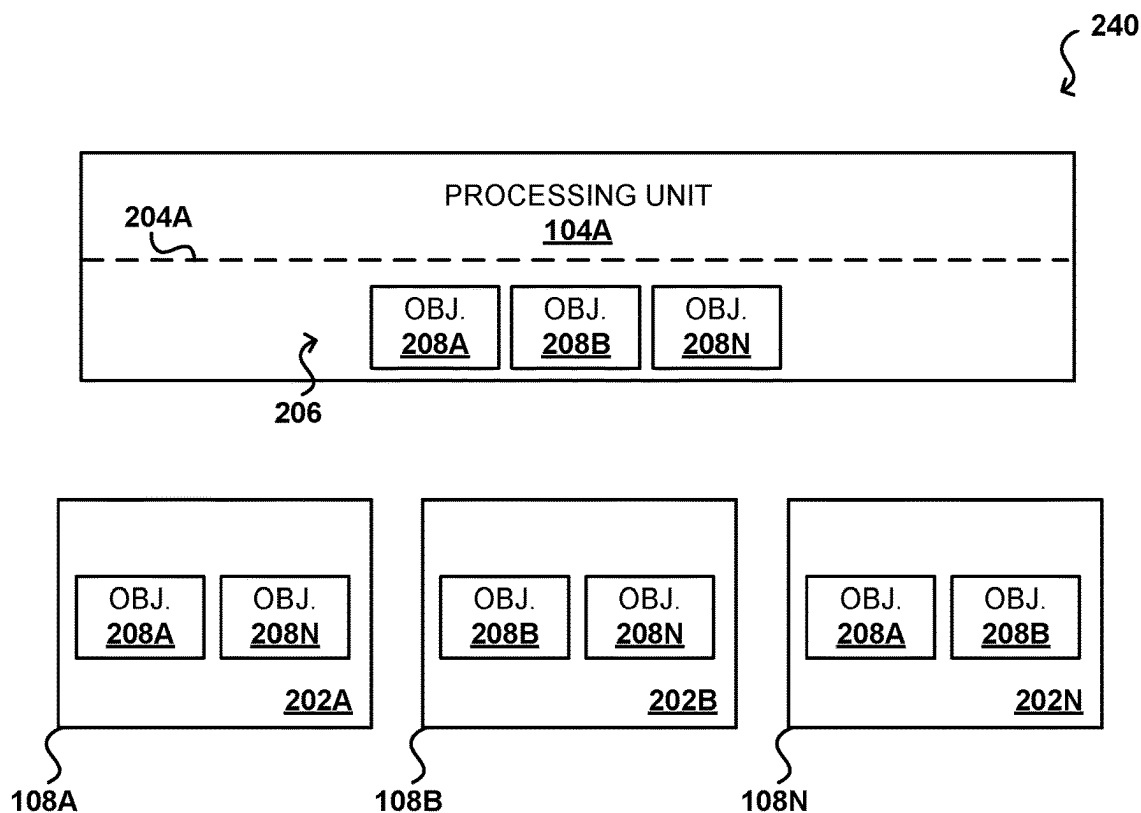


FIG. 2C

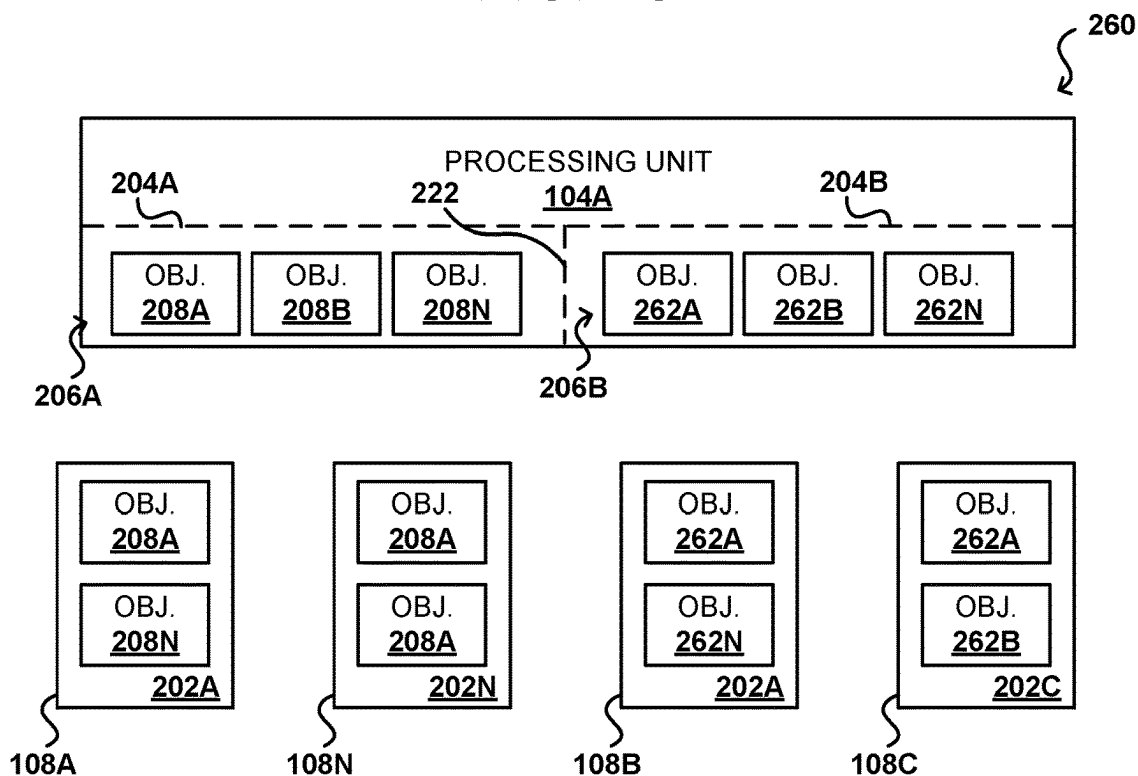


FIG. 2D

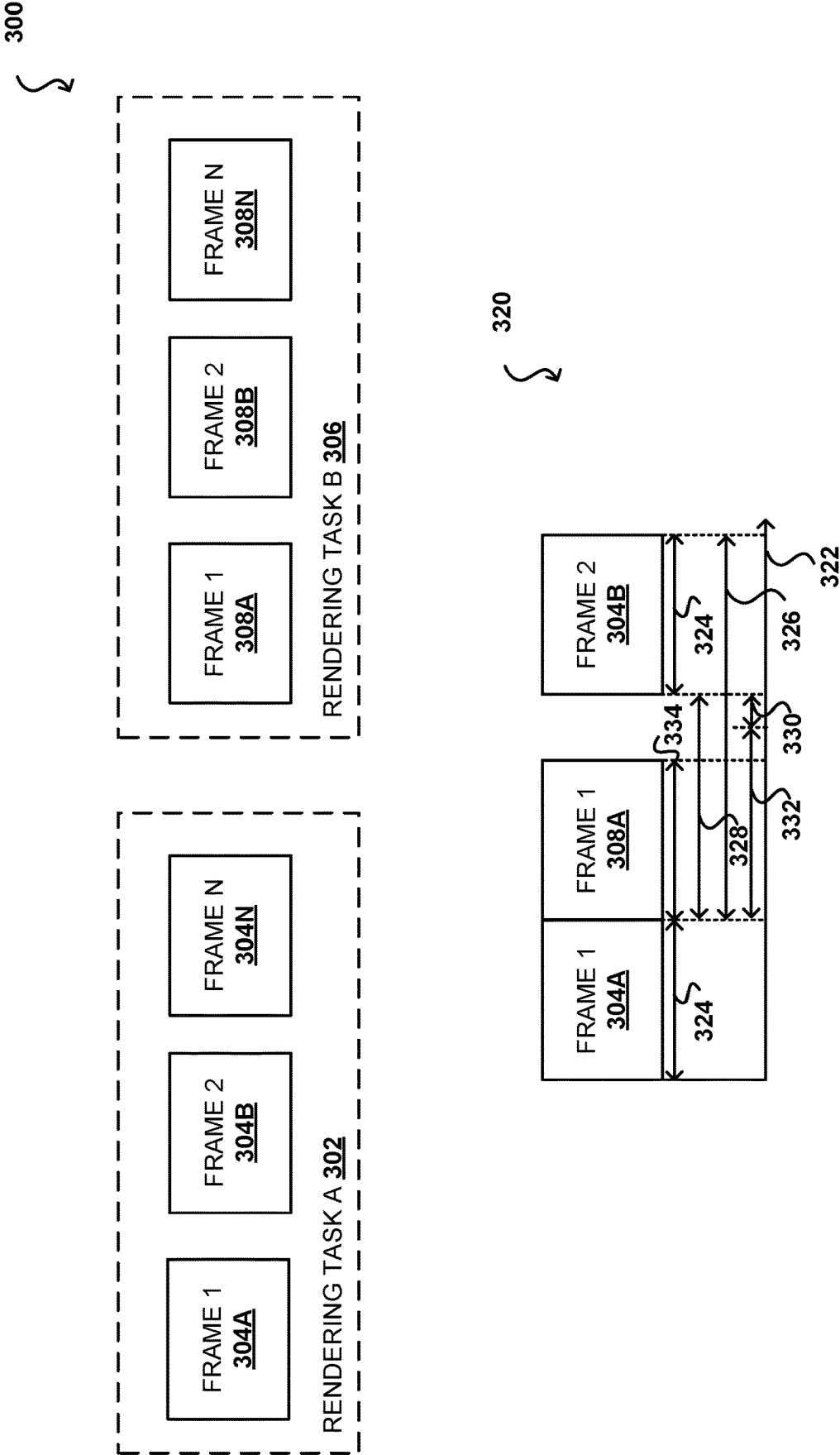


FIG. 3

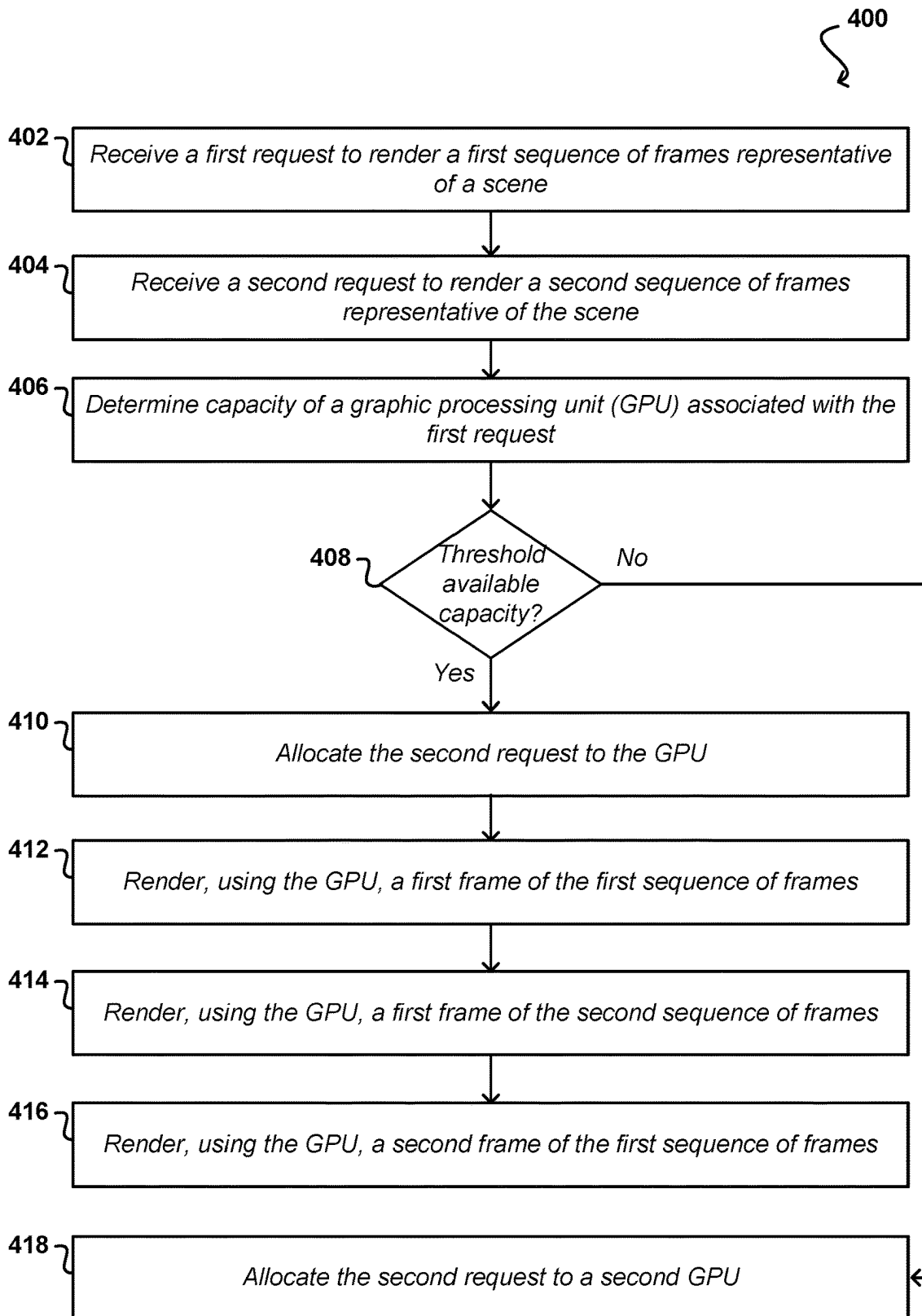


FIG. 4A

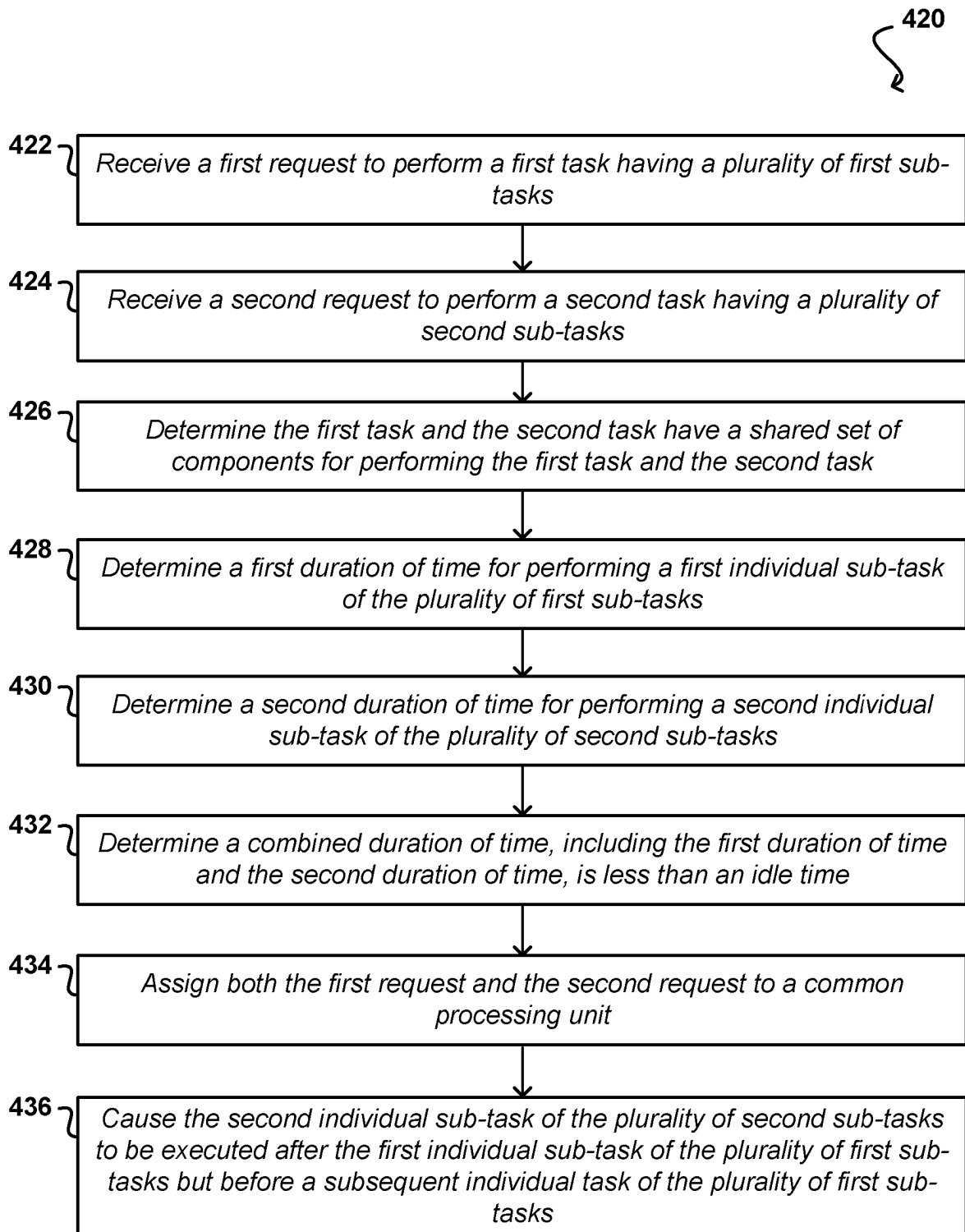


FIG. 4B

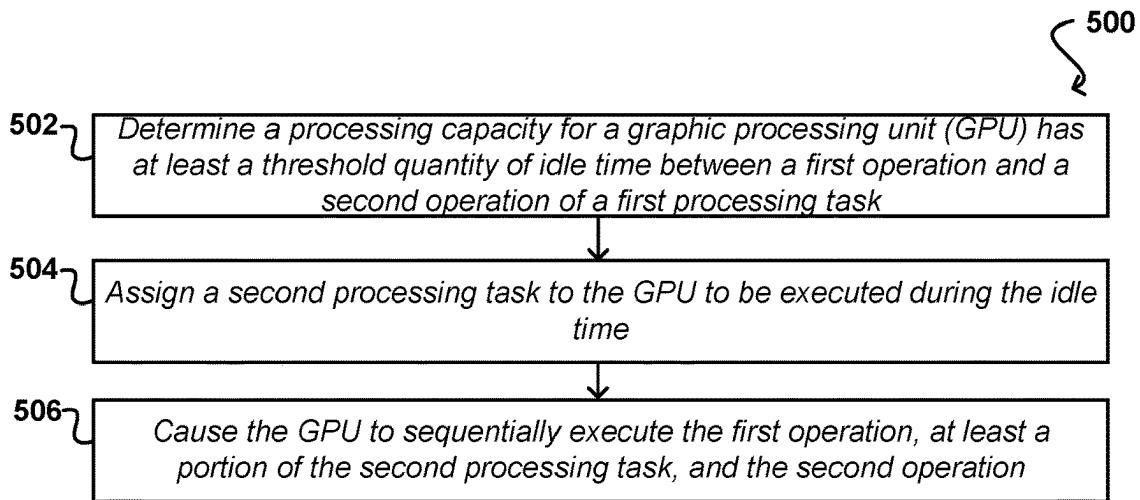


FIG. 5A

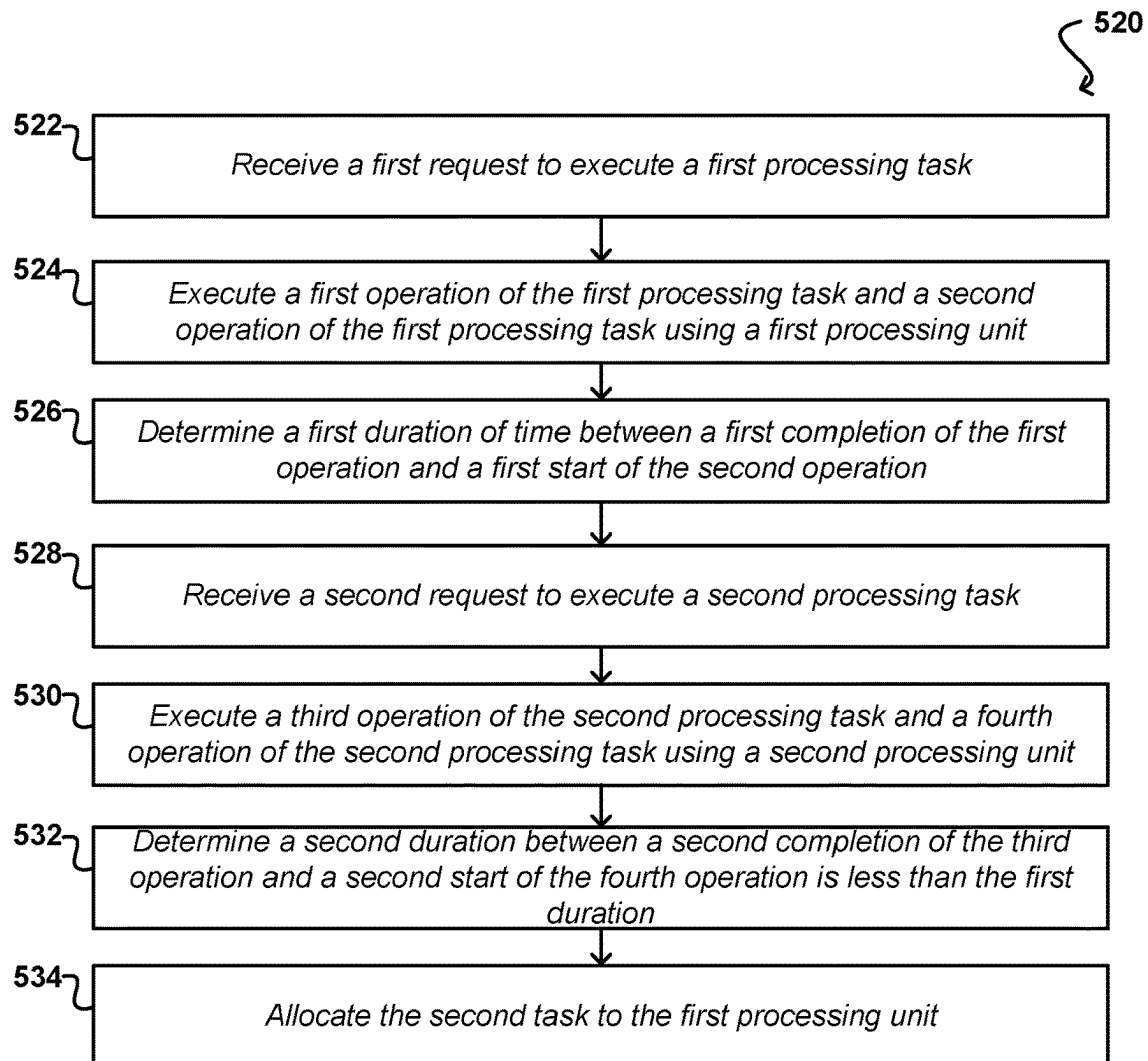


FIG. 5B

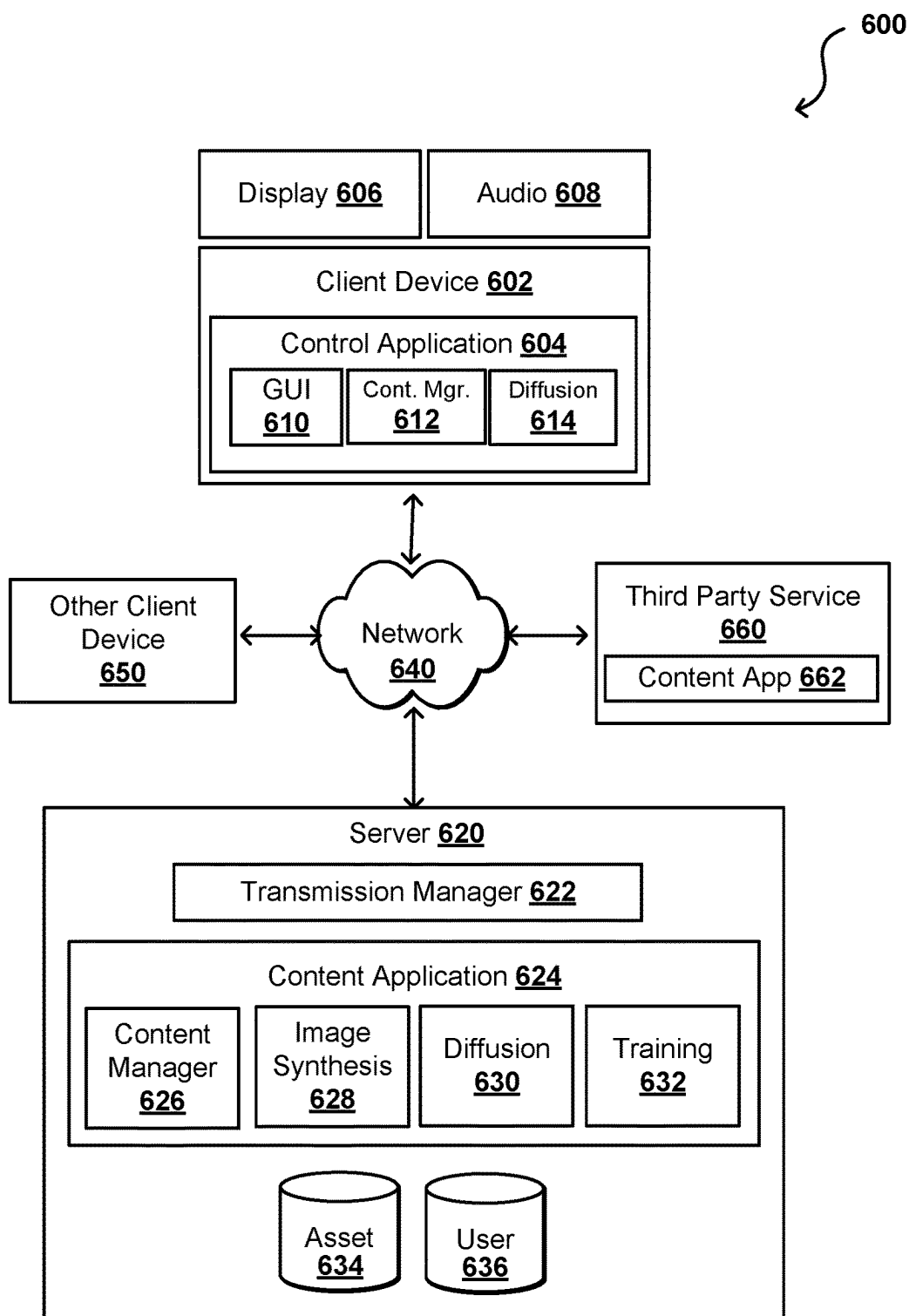


FIG. 6

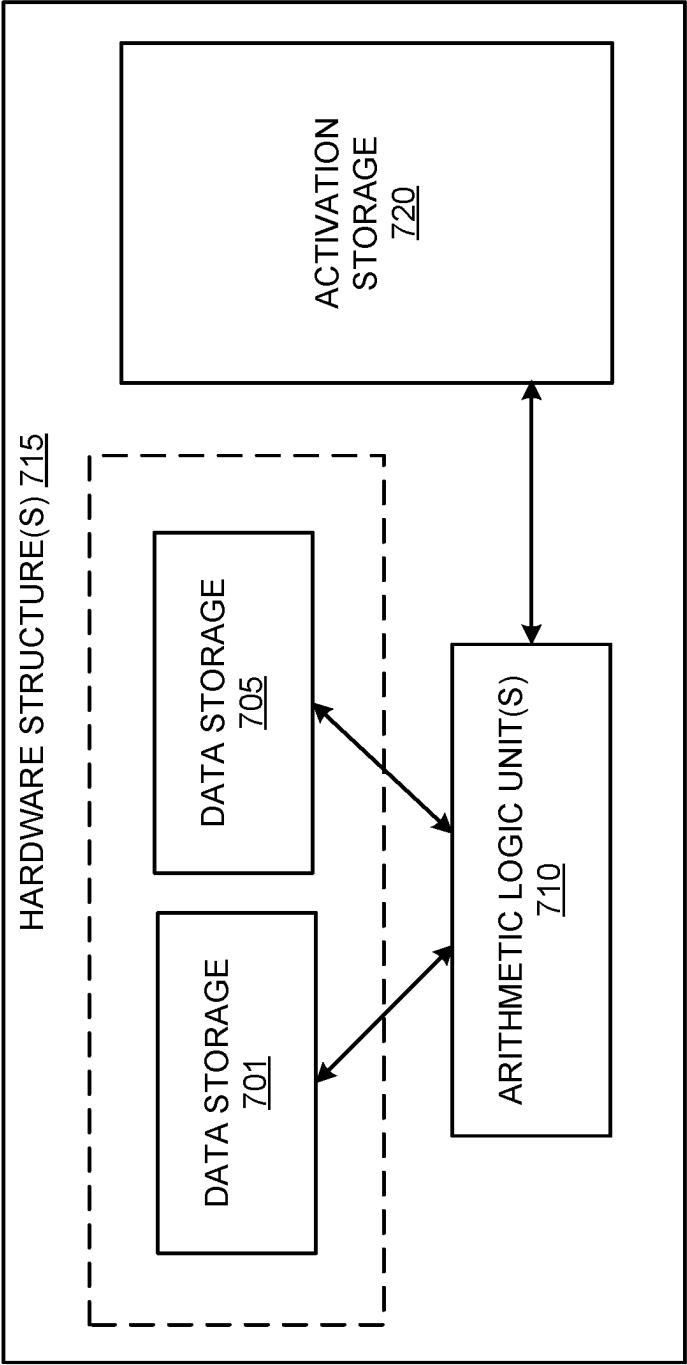


FIG. 7A

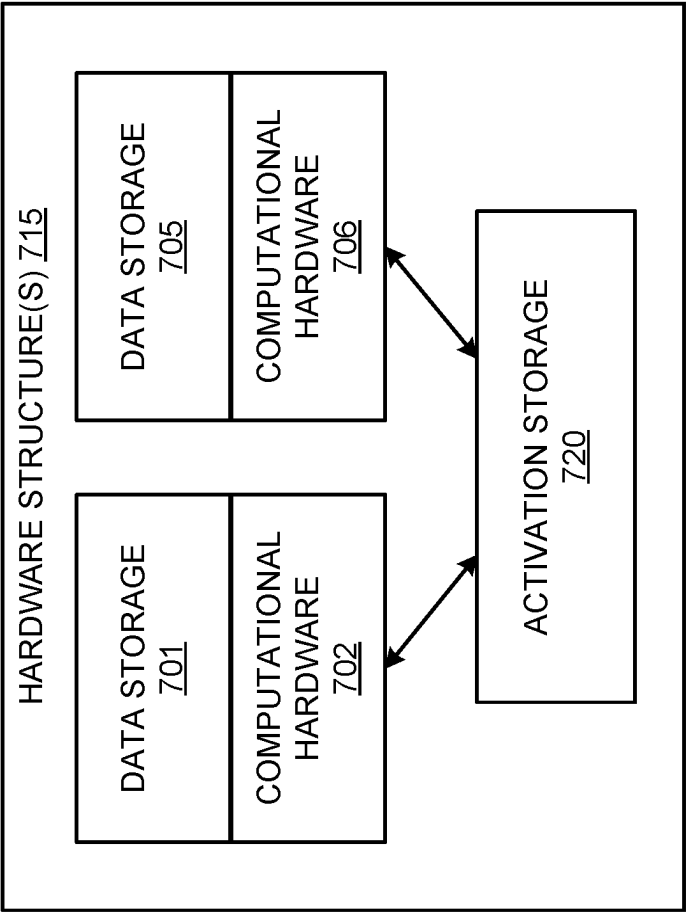


FIG. 7B

DATA CENTER
800

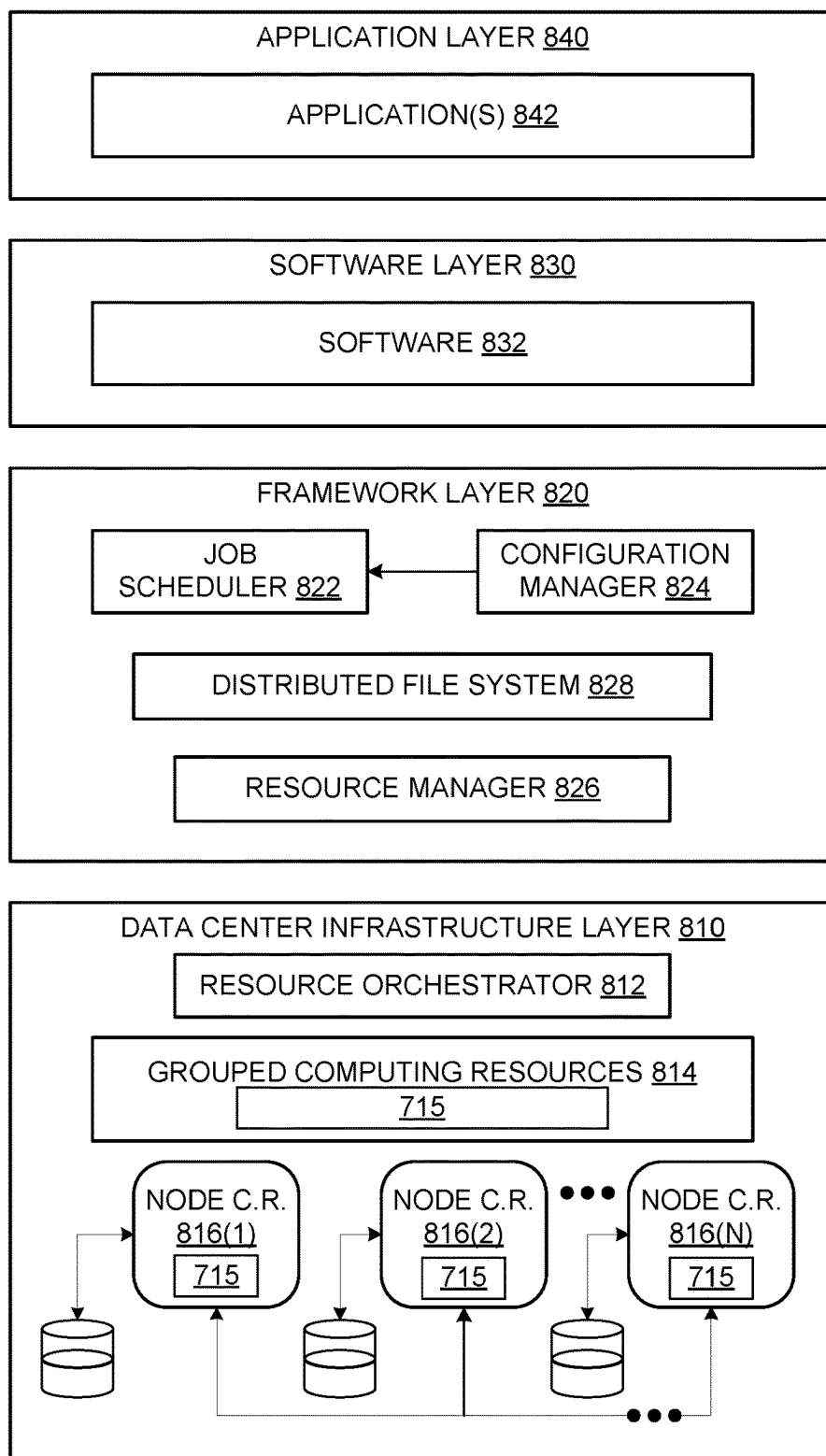


FIG. 8

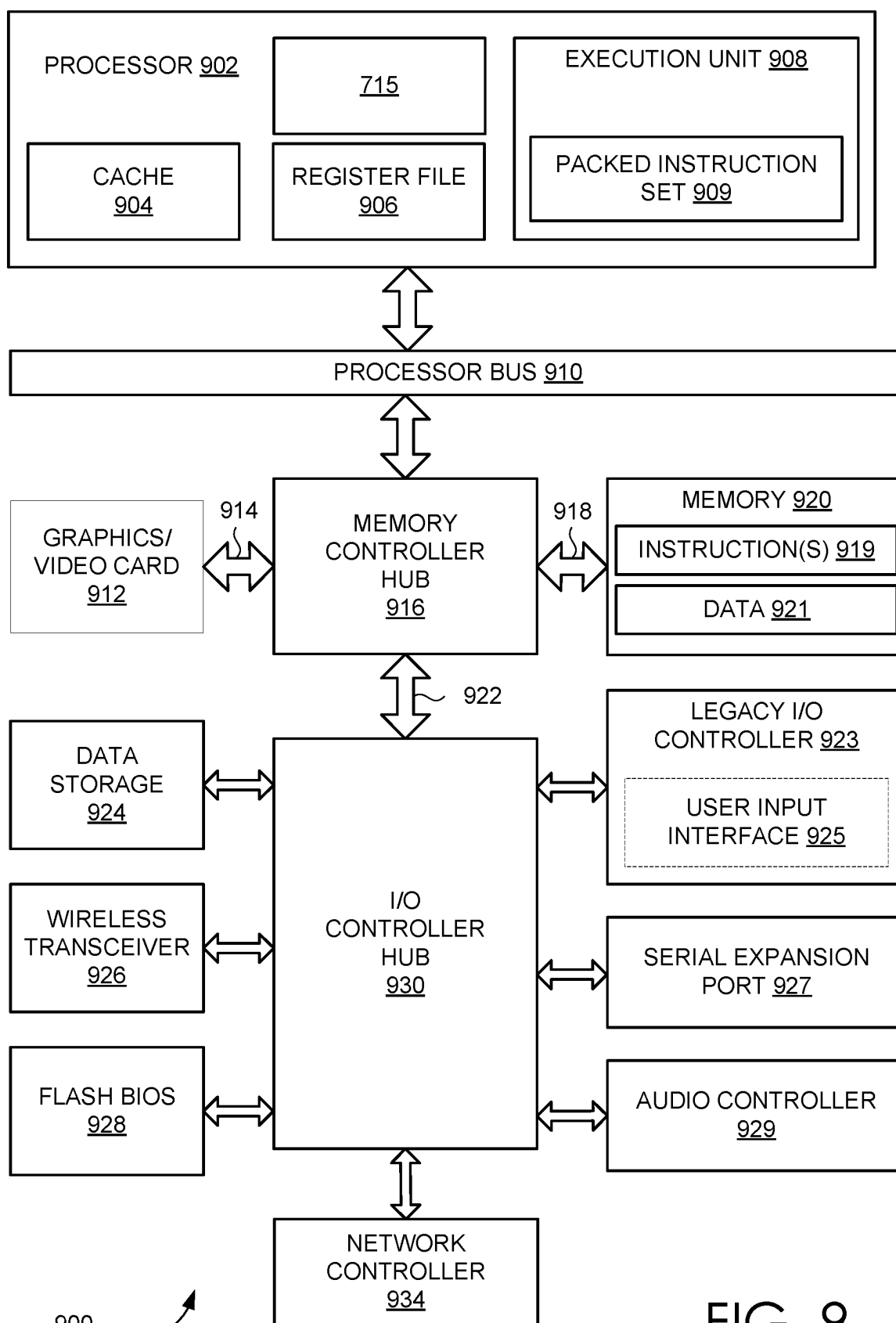


FIG. 9

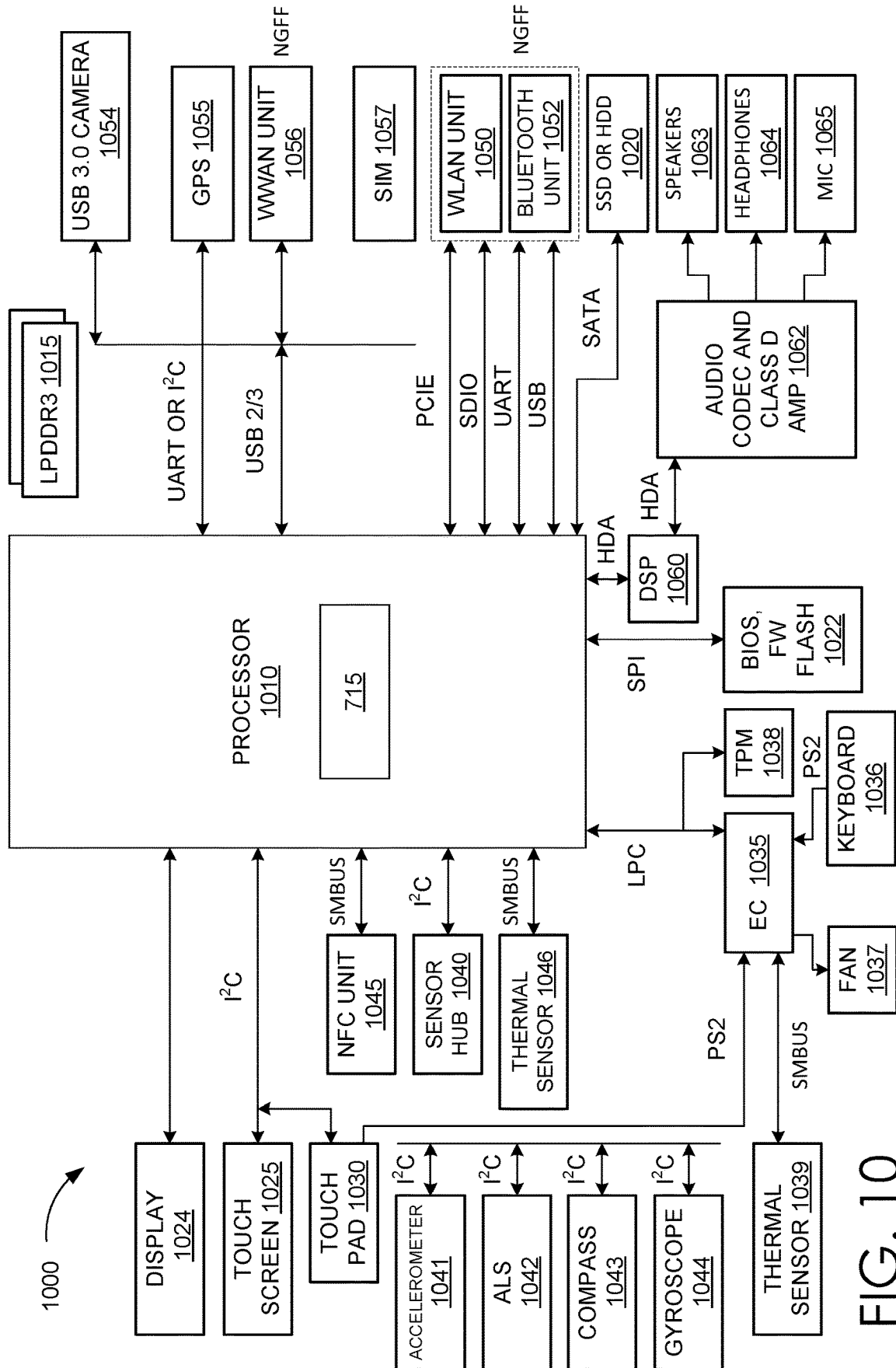


FIG. 10

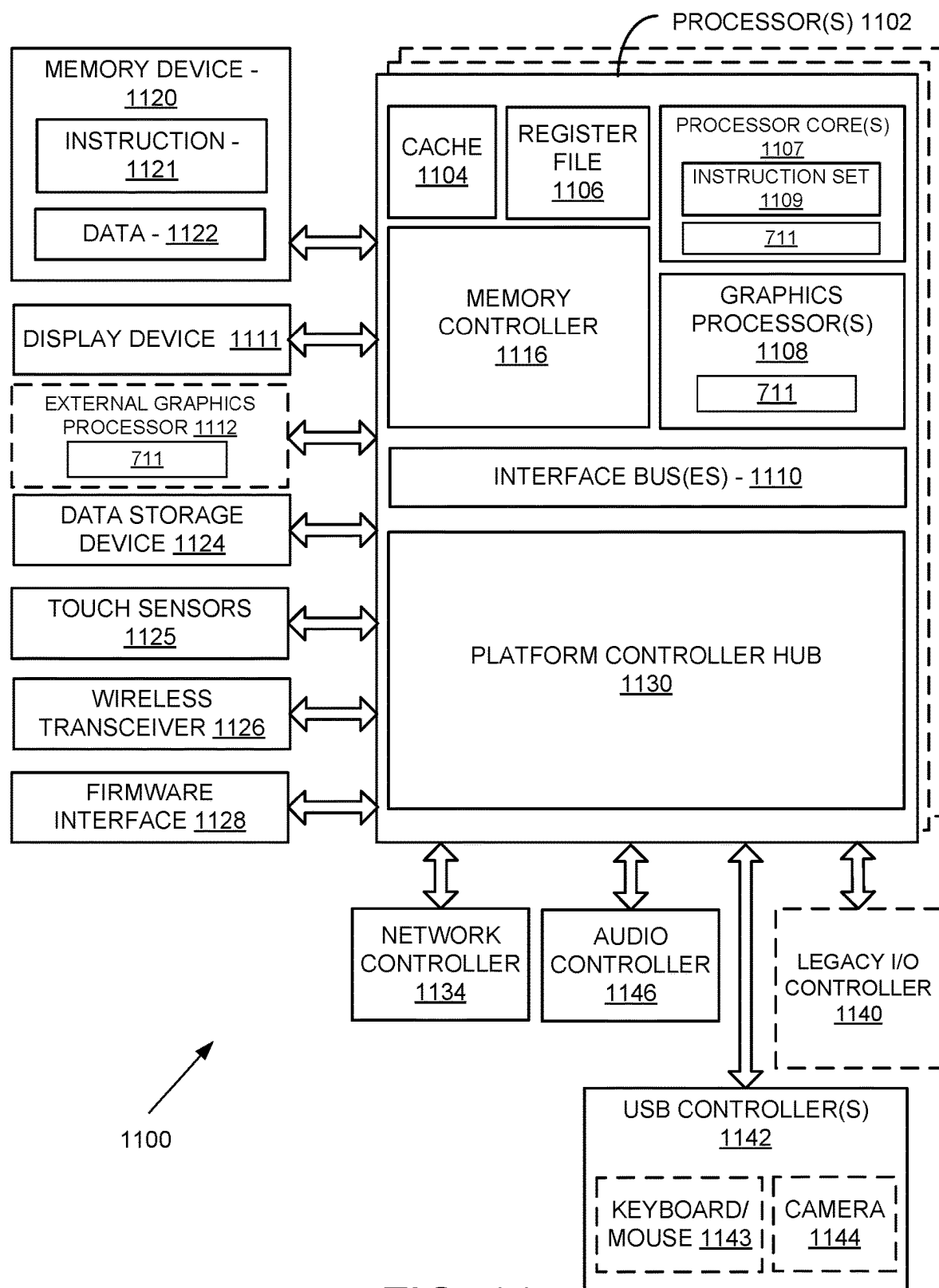


FIG. 11

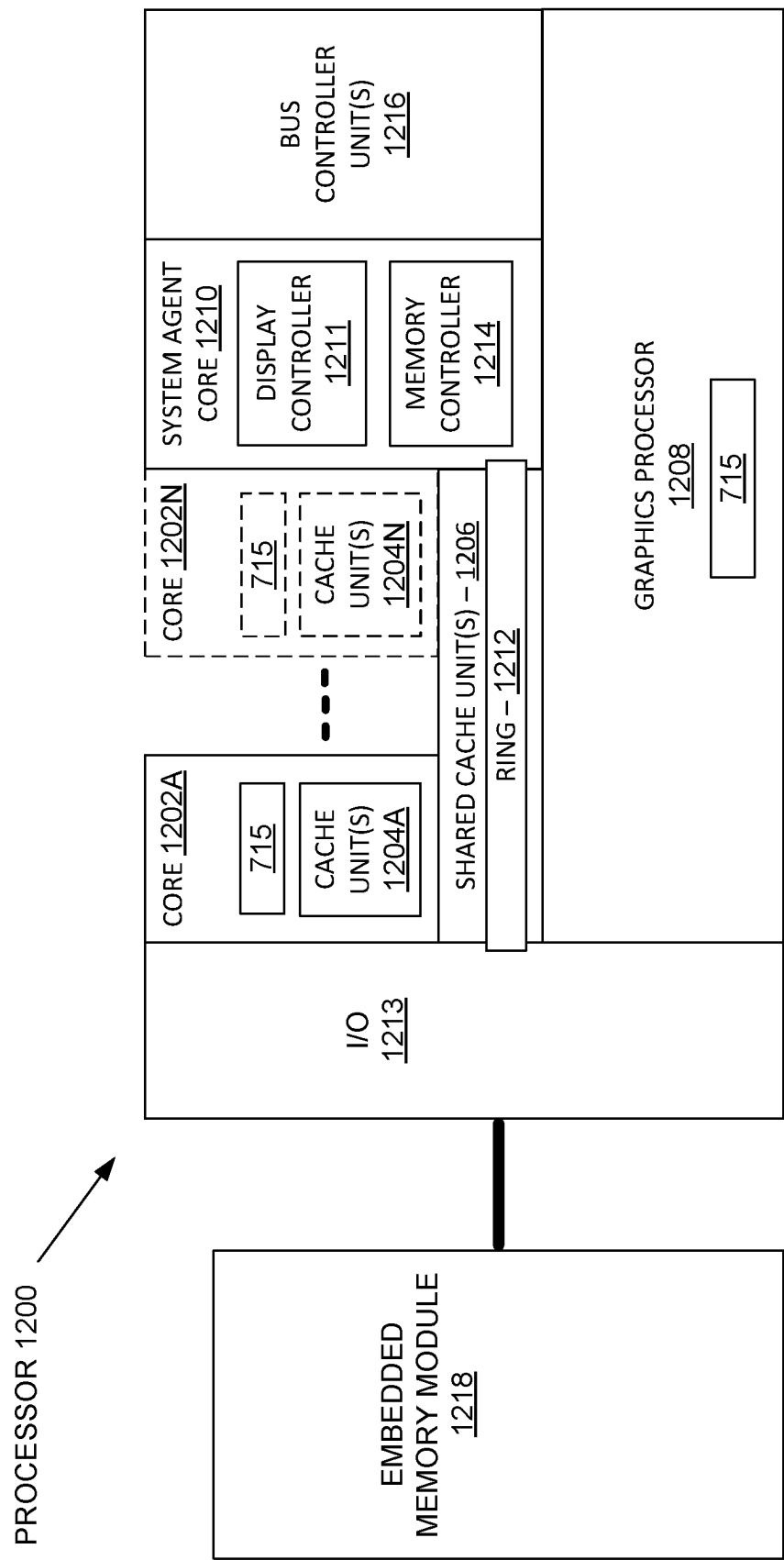


FIG. 12

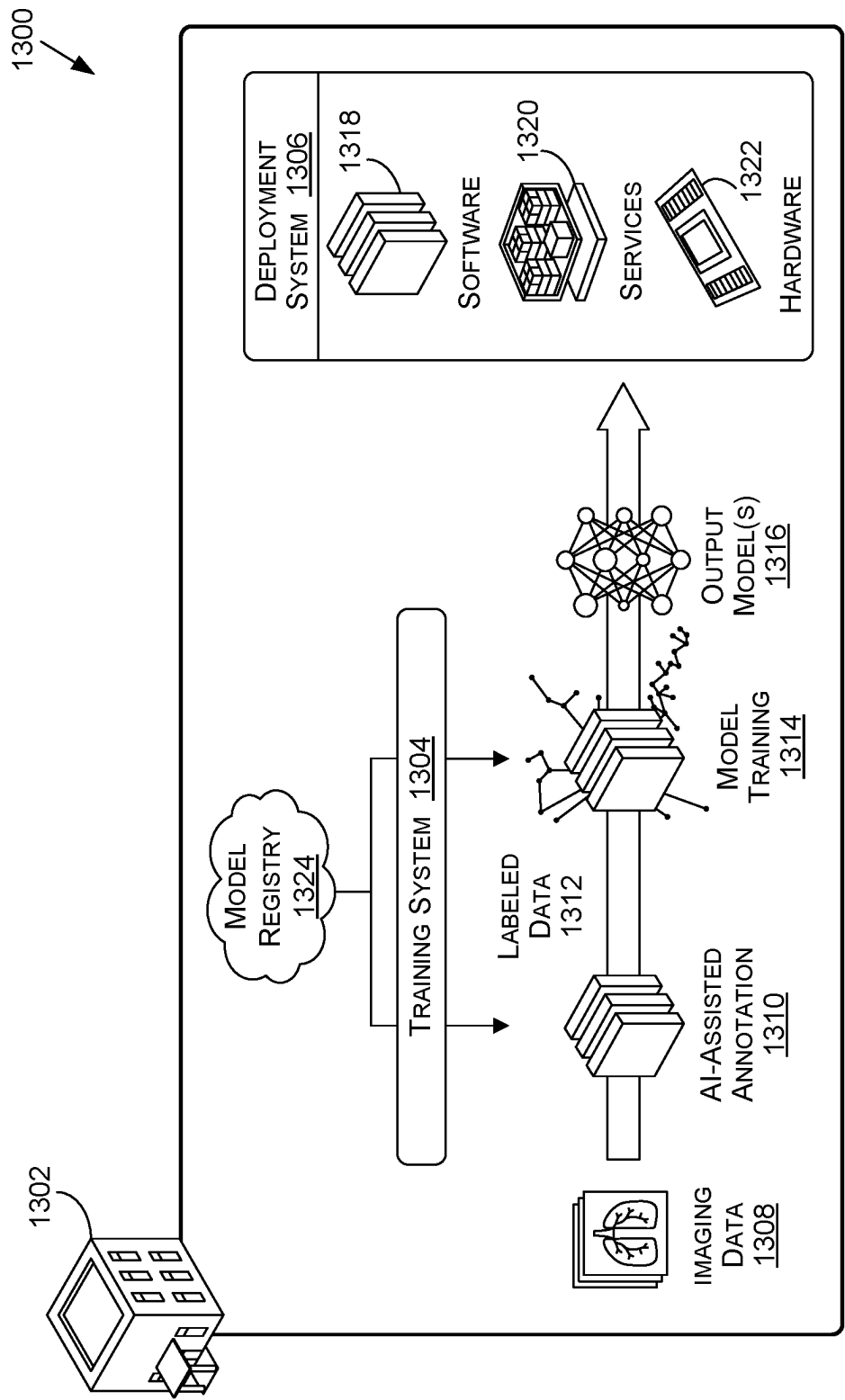


FIG. 13

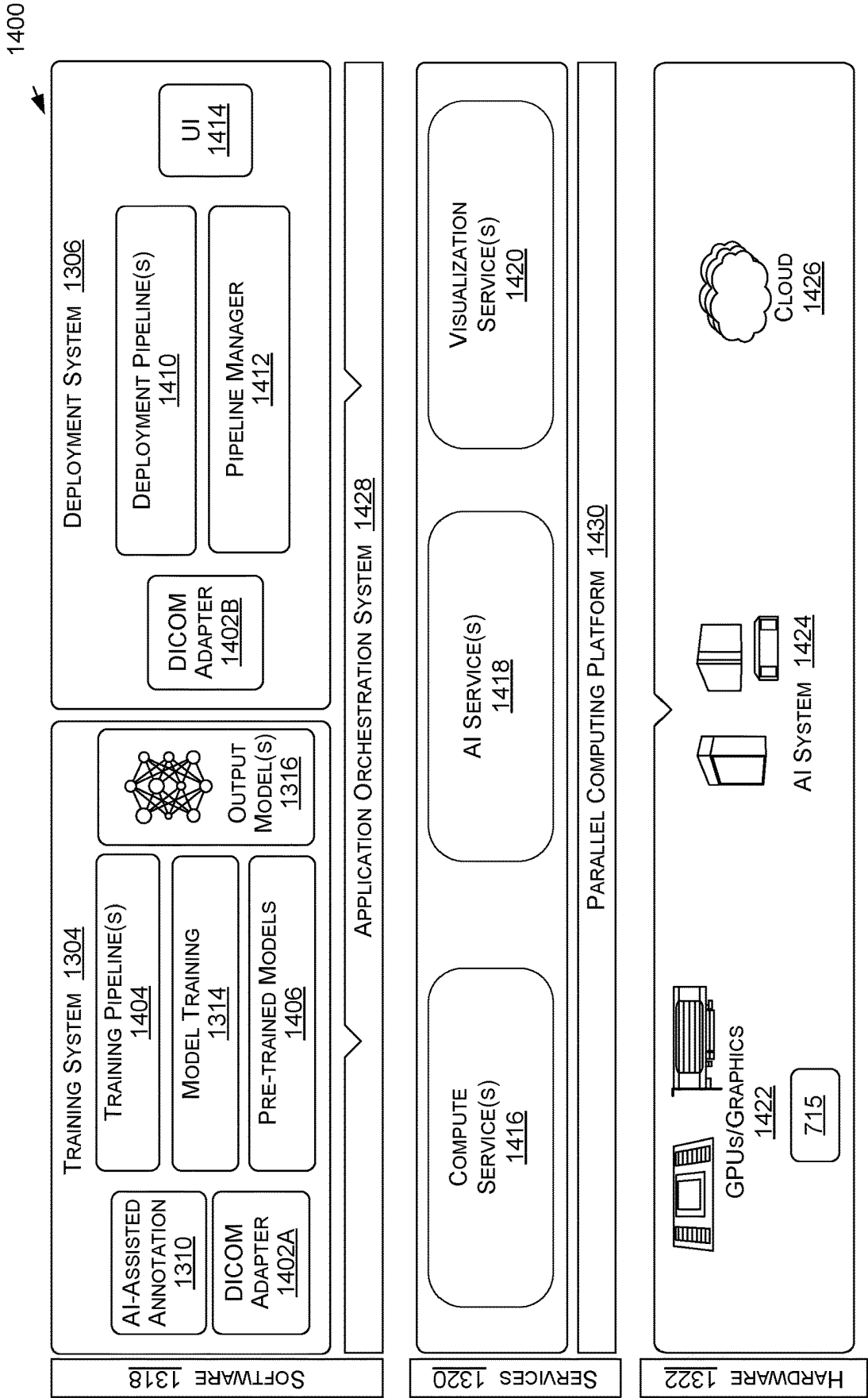


FIG. 14

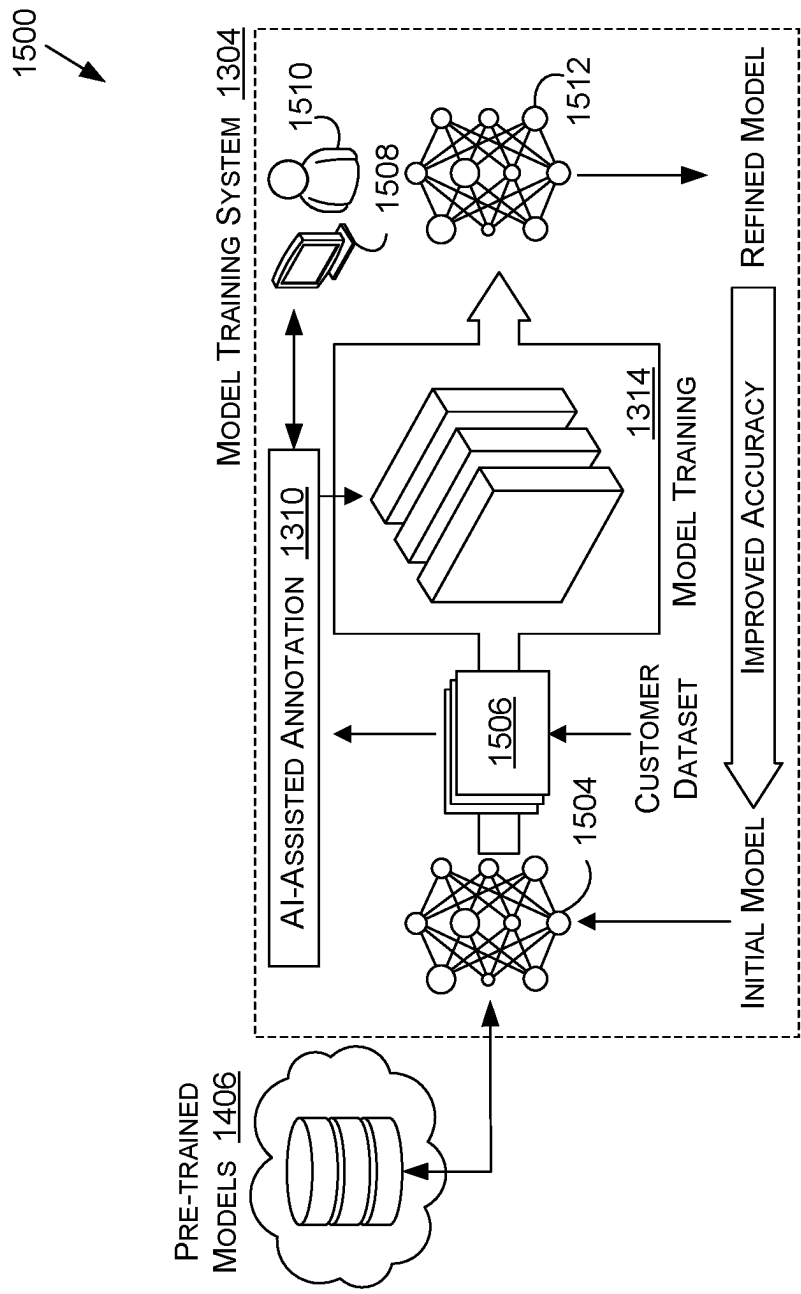


FIG. 15A

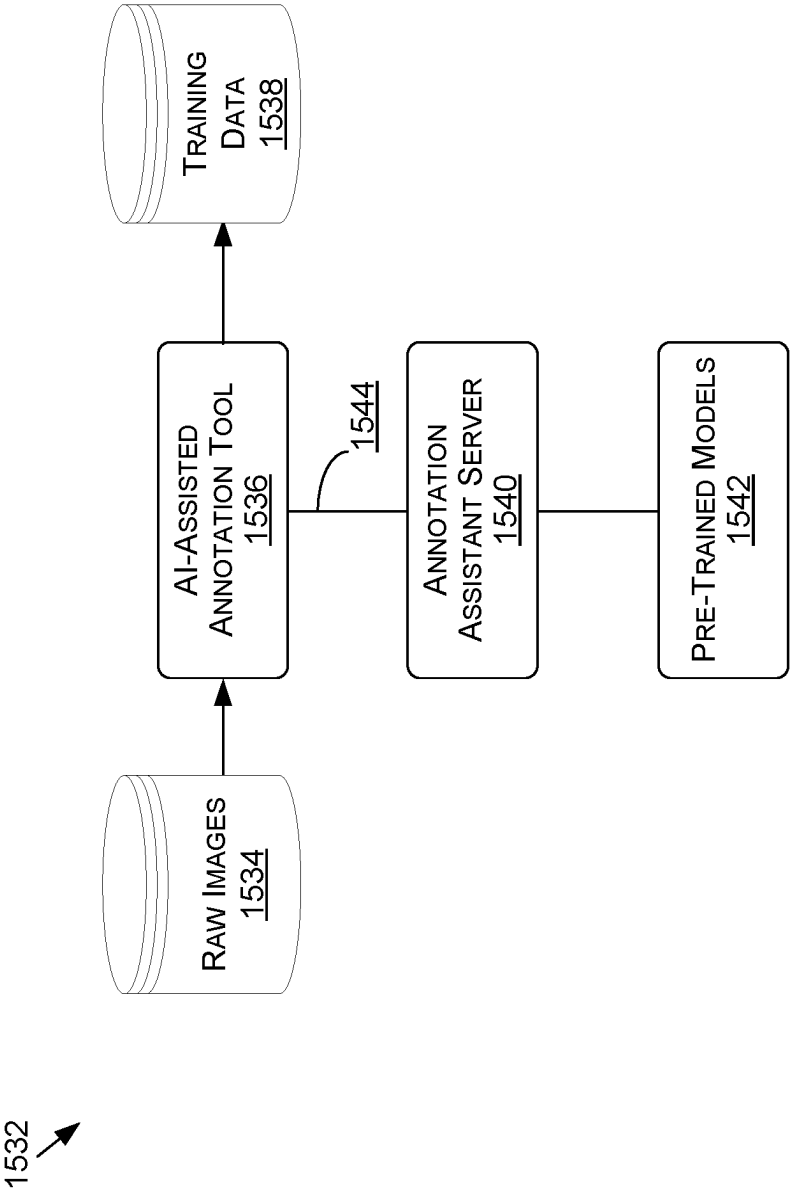


FIG. 15B

**DYNAMIC SHARING OF GRAPHICS
PROCESSING UNIT (GPU)
COMPUTATIONAL CAPABILITIES BASED
ON PROCESSING DENSITY**

BACKGROUND

[0001] A variety of computing applications may use processing units, such as graphics processing units (GPUs) or central processing units (CPUs), to provide different compute services for users. For example, GPUs can be used in interactive visualization services where a number of clients may access one or more scenes that consist of a predefined set of models, materials, animations, and the like. To support the services, groups of GPUs may be used to render different visualizations responsive to requests from a number of users. Typically, as the number of clients requesting use of the services increases, so does the number of GPUs necessary to provide the service. In an effort to provide improved scaling, GPUs may be virtually (e.g., logically) subdivided to allocate fractions of GPUs to different clients. However, GPUs can only be subdivided a limited number of times until the remaining processing capabilities are insufficient for the specified tasks. As a result, scaling to meet a large number of clients is costly.

BRIEF DESCRIPTION OF THE DRAWINGS

[0002] Various embodiments in accordance with the present disclosure will be described with reference to the drawings, in which:

[0003] FIG. 1 illustrates an example environment for executing compute operations, in accordance with various embodiments;

[0004] FIGS. 2A-2D illustrate example configurations for allocation processing capabilities for processing units, in accordance with various embodiments;

[0005] FIG. 3 illustrates an example configuration for performing sequential processing tasks using a common processing unit, in accordance with various embodiments;

[0006] FIG. 4A illustrates an example process for allocating and executing processing tasks, in accordance with various embodiments;

[0007] FIG. 4B illustrates an example process for allocating and executing processing tasks, in accordance with various embodiments;

[0008] FIG. 5A illustrates an example process for allocating and executing processing tasks, in accordance with various embodiments;

[0009] FIG. 5B illustrates an example process for allocating and executing processing tasks, in accordance with various embodiments;

[0010] FIG. 6 illustrates components of a distributed system that can be utilized to update or perform inferencing using a machine learning model, according to at least one embodiment;

[0011] FIG. 7A illustrates inference and/or training logic, according to at least one embodiment;

[0012] FIG. 7B illustrates inference and/or training logic, according to at least one embodiment;

[0013] FIG. 8 illustrates an example data center system, according to at least one embodiment;

[0014] FIG. 9 illustrates a computer system, according to at least one embodiment;

[0015] FIG. 10 illustrates a computer system, according to at least one embodiment;

[0016] FIG. 11 illustrates at least portions of a graphics processor, according to one or more embodiments;

[0017] FIG. 12 illustrates at least portions of a graphics processor, according to one or more embodiments;

[0018] FIG. 13 is an example data flow diagram for an advanced computing pipeline, in accordance with at least one embodiment;

[0019] FIG. 14 is a system diagram for an example system for training, adapting, instantiating and deploying machine learning models in an advanced computing pipeline, in accordance with at least one embodiment; and

[0020] FIGS. 15A and 15B illustrate a data flow diagram for a process to train a machine learning model, as well as client-server architecture to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment.

DETAILED DESCRIPTION

[0021] In the following description, various embodiments will be described. For purposes of explanation, specific configurations and details are set forth in order to provide a thorough understanding of the embodiments. However, it will also be apparent to one skilled in the art that the embodiments may be practiced without the specific details. Furthermore, well-known features may be omitted or simplified in order not to obscure the embodiment being described.

[0022] The systems and methods described herein may be used by, without limitation, non-autonomous vehicles or machines, semi-autonomous vehicles or machines (e.g., in an in-cabin infotainment or digital or driver virtual assistant application)), autonomous vehicles or machines, piloted and un-piloted robots or robotic platforms, warehouse vehicles, off-road vehicles, vehicles coupled to one or more trailers, flying vessels, boats, shuttles, emergency response vehicles, motorcycles, electric or motorized bicycles, aircraft, construction vehicles, trains, underwater craft, remotely operated vehicles such as drones, and/or other vehicle types. Further, the systems and methods described herein may be used for a variety of purposes, by way of example and without limitation, for machine control, machine locomotion, machine driving, synthetic data generation, model training or updating, perception, augmented reality, virtual reality, mixed reality, robotics, security and surveillance, simulation and digital twinning, autonomous or semi-autonomous machine applications, deep learning, environment simulation, object or actor simulation and/or digital twinning, data center processing, conversational artificial intelligence (AI), generative AI with large language models (LLMs), light transport simulation (e.g., ray-tracing, path tracing, etc.), collaborative content creation for 3D assets, cloud computing and/or any other suitable applications.

[0023] Disclosed embodiments may be comprised in a variety of different systems such as automotive systems (e.g., a control system for an autonomous or semi-autonomous machine, a perception system for an autonomous or semi-autonomous machine), systems implemented using a robot, aerial systems, medial systems, boating systems, smart area monitoring systems, systems for performing deep learning operations, systems for performing simulation operations, systems for performing digital twin operations, systems implemented using an edge device, systems incor-

porating one or more virtual machines (VMs), systems for performing synthetic data generation operations, systems implemented at least partially in a data center, systems for performing conversational AI operations, systems for performing generative AI operations using LLMs, systems for performing light transport simulation, systems for performing collaborative content creation for 3D assets, systems implemented at least partially using cloud computing resources, and/or other types of systems.

[0024] Approaches in accordance with various embodiments can be used to increase graphics processing unit (GPU) density by allowing multiple users to share GPU processing capabilities during GPU idle times. Systems and methods are directed toward dynamic GPU sharing by allowing multiple users to receive an output performance at a given rate using a common GPU. For example, with rendering operations, a user may be interested in receiving a certain frame rate as an output. For a given scene, the GPU may be capable of processing more than the frame rate expected by the users. During downtimes between providing the frames to a first user, processing capabilities may be used by a second user. The second user may be rendering a frame that uses a common scene as the first user so that background information is also loaded onto and shared by the GPU. In this manner, idle times may be filled with processing for multiple users, and as a result, fewer GPUs may be needed to support multiple users for a given scene. GPU utilization may be monitored and controlled statically or dynamically, with different thresholds being used to determine whether a given GPU has capacity to add users or if users should be moved to different GPUs.

[0025] Various embodiments of the present disclosure are directed toward overcoming problems associated with scaling processing capabilities, such as for providing various services to one or more clients within a distributed computing environment. Using the example of interactive visualization services, one or more clients may access different scenes consisting of a predefined set of models, materials, animations, and/or the like and processing (e.g., rendering) the environment may be executed on a GPU one frame at a time with a set of frames being provided sequentially to the user. Typically, processing units, such as GPUs, are allocated on a time or resource availability basis (e.g., processing for a given period of time for a first user, processing for a given job, etc.). In other words, GPUs are often considered an exclusive resource due to GPU abilities to enable parallel processing. Systems and methods of the present disclosure are directed toward allocation of processing capabilities on a per-work unit associated with a given task, such as rendering. As a result, various embodiments use idle time between rendering of sequential frames in order to permit multiple tasks to be executed by the same GPU during times when the GPU would otherwise be idle. Systems and methods are directed toward dynamic sharing of resources instead of static splitting, which as noted herein, may still fail to fully utilize GPU processing capabilities.

[0026] Systems and methods may provide a dynamic monitoring system that may be used to evaluate an expected idle time or available capacity for a GPU executing one or more processing tasks and then assign one or more additional processing tasks to that GPU if the available capacity is greater than a threshold. In this manner, computational density may be increased for a given GPU (or for a set of GPUs) without increasing a total number of GPUs used to

execute various tasks. In at least one embodiment, one or more GPU parameters may be used to estimate available capacity based on a scheduled task for the GPU. The scheduled task may be separated or divided into different computations and/or outputs, where the computations have a scheduled, sequential processing order. Thereafter, availability between the sequential tasks may be determined, and if that availability exceeds a threshold, a second task with one or more computations and/or outputs that can be completed within the period of time associated for availability may be assigned to the same GPU to be performed between the sequential tasks of the first task already assigned to the GPU. Such a process does not “split” or otherwise divide the GPU statically, but rather, uses time that would otherwise be lost or spent not processing to perform additional tasks. Accordingly, systems and methods enable scaling more efficiently than adding GPUs or statically dividing GPUs for individual clients.

[0027] Various other such functions can be used as well within the scope of the various embodiments as would be apparent to one of ordinary skill in the art in light of the teachings and suggestions contained herein.

[0028] FIG. 1 illustrates an example environment 100 that may be used with embodiments of the present disclosure. In this example, a resource environment 102 is associated with a plurality of compute resources or processing units 104, such as a series of GPUs, that are arranged within a rack 106. The resource environment 102 may be associated with a data center or other compute center in which processing units may be rented, such as within a distributed computing environment (e.g., a “cloud” environment). In this arrangement, one or more providers may not own the underlying hardware (e.g., GPUs, CPUs, etc.) and may rent or otherwise contract for use of different computing capabilities. The providers may then instruct the resource environment 102 on what tasks to execute on their assigned compute resources 104. In at least one embodiment, the providers may rent compute resources 104 based on a variety of factors or metrics, such as expected load, and thereafter may increase or decrease a resource allocation based on trends or use.

[0029] In operation, one or more clients 108A-108N (e.g., users) may submit requests over one or more networks 110 to access content 112, which may be stored remotely from the resource environment 102 but is also accessible and responsive to one or more requests. Additionally, portions of the content 112 may be stored within the resource environment 102 and/or with an associated resource environment 102. Moreover, different portions of the content 112 may be stored and then deleted as a user interacts with the environment. The clients 108A-108N may be represented by one or more client devices (e.g., client computing devices), which may serve as a proxy to the user by making requests responsive to one or more input commands. Additionally, a client may navigate to one or more applications or access points using the device to submit a content request, among other options. As another example, a request may be transmitted as part of an automated or semi-automated workflow, which may or may not receive user interaction input. Accordingly, one or more client computing devices, associated with the clients 108A-108N, may be used with direct input from one or more users, from stored software instructions, from executions of various workflows, or combinations thereof. In various embodiments, the clients 108A-108N submit requests for use of compute resources on

behalf of a provider or other service that uses the resource environment **102**. For example, a provider may host a website and use the resource environment **102** to provide processing capabilities for one or more interactive features of the website. As a result, requests submitted by the clients **108A-108N** may be associated with an account for a provider.

[0030] In at least some embodiments, the request can include a request for content to be displayed on a computing device, and in many cases will include video, audio, animations, and/or the like or other media content that is transcoded for presentation on the device. The network(s) **110** can include any appropriate network, such as the Internet, a local area network (LAN), a cellular network, an Ethernet, or other such wired and/or wireless network. Furthermore, the resource provider **102** may include any appropriate resources for providing content and/or services from one or more third-party providers that use the services of the resource environment **102**, as may include various servers, data stores, and other such components known or used for providing content from across a network. In various embodiments, the client devices can be any appropriate computing or processing device, as may include a desktop or notebook computer, smartphone, tablet, wearable computer (i.e., smart watch, glasses, or contacts), set top box, kiosk, interactive display, or other such system or device.

[0031] One or more requests may be submitted over the network **110** to be received at the resource environment **102**, for example at an interface layer that may serve as a landing page or application program interface (API) to access different resources and/or content elements. User credentials may be checked prior to granting access to the content or resources within the resource environment **102**, for example using one or more user datastores **114** that may store user information for a given service associated with the resource environment **102**. As noted herein, the user datastores **114** may also be remotely accessible by the resource environment **102** and, as a result, may not be stored within the resource environment **102**. The user datastores **114** may be associated with the providers of particular services, the individual clients **108A-108N**, and/or combinations thereof. In this example, the resource environment **102** includes both physical components (e.g., the rack **106** and the processing units **104**) as well as representations of different systems, subsystems, and/or engines that may execute within or associated with the resource provider environment **102**, for example using the processing units **104**.

[0032] In at least one embodiment, a request manager **116** receives requests provided to the resource environment **102**. Example, non-limiting requests include requests to view and/or interact with content or to execute one or more compute functions, among other options. The request manager **116** may determine, for example using the user datastores **114**, whether the request submitter is authorized to perform the functions associated with the request and then may distribute one or more commands to permit access. For example, if the request is a content request, the request manager **116** may receive the request, authenticate the user, and then determine how to satisfy the request, such as allocating at least a portion of processing capabilities from one or more processing units **104** to execute the request. A compute predictor **118** and assignor **120** may be used to determine the set of resources that are allocated responsive to the request and then to assign the request to particular

resources. By way of example, the request may be associated with viewing and/or interacting with content **112**, and as a result, features or information associated with the content **112** may be obtained. The features or information may include data such as a frame rate for the content, information regarding objects making up the content (e.g., models, textures, etc.), and/or the like. As such, one or more predictions can be made regarding processing capabilities to provide the content in accordance with one or more metrics for content presentation. That is, the information may be used to determine a minimum amount of processing capability (within some threshold) to provide an agreed level of service and/or content presentation. As a result, different levels of service may lead to different predicted processing needs. By way of example, content that is very complex and provided in ultra-high definition will require more processing capability than simple content in standard definition. In this manner, the request may be allocated to one or more processing units **104** capable of providing the agreed upon and/or needed level of processing capabilities.

[0033] In at least one embodiment, the compute predictor **118** may determine one or more properties of the processing units **104** that will execute the request. For example, with respect to frame rate, the compute predictor **118** may determine that the frame rate for the request is 60 frames per second (fps) at a given frame size, resolution, etc. Therefore, the processing unit needs to be able to render a frame approximately every 16.67 milliseconds (ms). Often, the processing units **104** are capable of rendering individual frames much faster; for example, the processing units **104** may be able to render at 144 fps. Accordingly, the request may be assigned to a processing unit **104** that has capabilities that exceed its need, and as a result, will have idle time between different processing tasks or portions of tasks. As noted herein, systems and methods of the present disclosure may identify these idle times and assign additional processing tasks to fill the idle times without splitting or otherwise statically dividing the processing units **104**. Additionally, in at least one embodiment, the compute predictor **118** may also be used to monitor processing for a given task for a period of time and then determine an idle time between different sequential tasks. For example, a request may be received and allocated to a given processing unit **104** and, after a period of time, idle times between different portions of the overall task may be determined and used to allocate additional tasks for additional users to the same processing unit **104**.

[0034] When determining how to allocate or assign processing tasks, one or more features or properties of the processing units **104** may also be used, for example, by accessing information from one or more processing unit datastores **122**. The processing unit datastores **122** may include information regarding capabilities of the processing units **104**, scheduled use or allocation of the processing units **104**, and/or the like. Systems and methods may collect and analyze processing unit information along with information about the request to predict computational needs and then assign different tasks to different processing units **104** according to those needs.

[0035] In operation, processing may be monitored over time, for example, by using one or more telemetry units **124**. The telemetry units **124** may include sensors associated with the rack **106** and/or processing units **104** to determine a load. For example, temperature may be monitored to determine

whether loads should be decreased in order to maintain the processing units **104** within a specified range of operation. Additionally, in other embodiments, memory use or latency may be monitored to determine whether a minimum level of operability is being provided, which may lead to shifting or reassigning different processing tasks to different processing units **104**. In this manner, requests may be received, evaluated, assigned, and then monitored.

[0036] Embodiments of the present disclosure may be used to address and overcome problems and drawbacks of existing systems. As one non-limiting example of rendering content for a client within an interaction environment, a client may request access to the environment to interact with one or more content elements. The interaction may provide the user with opportunities to modify or change different objects within the content, such as modifying a color or changing a view direction, among other options. High fidelity rendering of the content elements may be important to the providers of such services, as they can be used to market or otherwise entice users to purchase goods or otherwise provide a favorable experience that may be beneficial for users.

[0037] Typically, the environments offer a substantial number of different configurations and options, thereby making it difficult to pre-render or prepare the number of potential combinations. Even to the extent that certain popular configurations can be prepared in advance, this may be a time consuming process that occupies substantial amounts of memory. Alternatively, content may be rendered in real or near-real time (e.g., without significant delay), which may be referred to as “on demand,” responsive to inputs provided by the interactions. The process of rendering on demand allows interactive visualization and increases the flexibility of the configuration in exchange for allocating computation resources for clients that are in process of configuring custom products.

[0038] Traditionally, the process of rendering can be split between multiple rendering units (e.g., processing units) in a manner that each of the units should be able to render any of the configurations to allow simple distribution of any of the customer requests to any rendering unit. While this arrangement works for smaller circumstances, scaling can become a significant problem. For example, when the number of clients is in the range of hundreds of thousands or millions, allocating a dedicated rendering unit (e.g., dedicated GPU or CPU) per user is prohibitively expensive and/or unrealistic. Therefore, instead of allocating a rendering unit per customer, a fraction of a rendering unit may be allocated to a particular customer. For example, GPUs may be divided as virtual GPUs, such as dividing the GPU into two, four, eight, or any reasonable number of parts. While allocating a fraction of the rendering unit per consumer provides a higher level of density and support for more customers at the same time, the rendering unit is shared between the jobs on predefined slices that are calculated as fraction of the whole unit. If the fraction of the unit is too small, the client cannot render very complex models. If the fraction of the unit is too big, the allocation keeps most of the rendering unit idle.

[0039] Various embodiments overcome these problems and others by leveraging shared stored information between different tasks to permit multiple clients to share a single processing unit to occupy idle times between tasks. For example, with on demand rendering, each processing unit

needs the background data to render all possible configurations. This information may be stored in processing unit memory and then accessed in response to input commands. Because multiple clients may be using the same interaction environment, with the same background information, systems and methods may enable these users to share one or more processing units that may access the same underlying information stored in a shared memory. Accordingly, instead of slicing the processing unit into smaller processing units, systems and methods instruct the processing unit to execute a first portion of a first task (such as rendering a single frame) for a first user and then executing a first portion of a second task second task (such as rendering a single frame) for a second user in the idle time between execution of a second portion of the first task (such as rendering a subsequent frame) for the first user. The parameters of the different processing tasks may define how many tasks can be fit within an idle time in order to increase a density of processing for a given processing unit without statically dividing the processing unit. Embodiments of the present disclosure therefore leverage the idle time and shared memory to provide improved processing density of GPUs without the drawbacks associated with static splitting and/or single processing unit allocations.

[0040] FIG. 2A illustrates an example configuration **200** in which a first processing unit **104A** is used to render a processing task for a first client **108A** and a second processing unit **104B** is used to render a processing task for a second client **108B**. In this example, client devices may be used that include respective displays **202A**, **202B** to render various objects and/or content elements associated with a particular rendering task. As shown, each individual processing task is divided between different processing units **104A**, **104B**. The processing units **104A**, **104B** each include memories **204A**, **204B** that are used to store task information **206**. As shown, there is equivalent task information **206** between both of the processing tasks associated with the different processing units **104A**, **104B**. For example, both the clients **108A**, **108B** may be accessing an interaction environment provided on a webpage where the clients **108A**, **108B** can make selections to customize their individual experiences, but where the underlying content is the same. For example, the task information **206** may include objects **208A-208N** that may be rendered on the displays **202A**, **202B**. In various embodiments, the underlying content may come from a common provider, such as a given interaction environment in which users can customize their own experience from a set of underlying components.

[0041] In this example, there is overlapping information between the two renderings provided on the displays **202A**, **202B**. For example, the first display **202A** includes the objects **208A** and **208N** and the second display **202B** also includes the object **208N**. Additionally, as noted, each of the respective memories **204A**, **204B** stores the same task information **206**. As noted herein, such a system may enable allocation of tasks for different clients, however, may not be scalable when the number of users increases significantly. Moreover, there are wasted resources due to the duplicative memory storages and wasted time between processing tasks. For example, if the parameters of the processing task are less than the capabilities of the processing units **104A**, **104B**, there may be time where the processing units **104A**, **104B** are sitting idle, which could otherwise be effectively used for additional tasks. Systems and methods overcome these

problems, and others, by permitting a shared memory space for a common underlying set of task information to be used to execute discrete tasks and/or portions of larger tasks, such as sequential processing tasks, in a manner that allows one task (or a portion of that task) to execute during an idle time for another task (or a portion of that task).

[0042] FIG. 2B illustrates an example configuration 220 in which the first processing unit 104A is divided or split 222 between different processing tasks for the first client 108A and the second client 108B. As noted herein, the processing unit 104A may contain the memory 204A, which in this example is split or divided between the different processing tasks for the different clients. The split 222 may also be referred to as a static split. While an example of splitting the processing unit 104A into two slices is shown, the illustrated configuration is provided by non-limiting example and any number of divisions or splits may be provided in order to statically divide the capabilities of the processing unit 104A between different clients.

[0043] The configuration 220 partially overcomes the scaling problem of the configuration 200 in that a single processing unit can be used to execute the two different tasks for the clients 108A, 108B. However, there is still lost efficiency and idle time that is not used to increase processing density of the processing unit 104A. For example, the task information 206 is still stored twice, one in the allocated memory for the first client 108A and once in the allocated memory for the second client 108B. Additionally, as noted herein, for complex tasks the slicing of processing units may lead to reduced performance and/or capabilities. Systems and methods address and overcome these problems by sharing the memory allocation between multiple tasks and timing execution of certain task portions for idle times.

[0044] FIG. 2C illustrates an example configuration 240 that may be used with embodiments of the present disclosure. As illustrated, the first processing unit 104A includes the memory 204A that stores the task information 206. In this example, each of the clients 108A-108N may be rendering a scene that includes the same underlying task information 206, and as a result, may be shared or otherwise reused when each of the clients 108A-108N provides inputs to personalize their own experiences with the content. For example, the first client 108A may wish to render a first set of objects 208A, 208N, the second client 108B may wish to render a second set of objects 208B, 208N, and the n-th client 108N may wish to render a third set of objects 208A, 208B. As will be described, rendering of individual frames for each of the clients 108A-108N may occur during idle times for as long as a window is available to execute additional tasks. For example, if a task associated with the first client 108A takes $\frac{1}{20}$ of a second, there is additional time to execute tasks for the second client 108B and the n-th client 108N prior to the need to perform subsequent tasks for the first client 108A. Furthermore, grouping clients 108A-108N by their underlying content environment shares the task information 206 within a common memory location to preserve resources. As a result, systems and methods may allocate and configure processing units to provide a minimum level of computation/output, rather than allocating a number of cores of theoretical processing capability.

[0045] FIG. 2D illustrates a configuration 260 that may be used with embodiments of the present disclosure in which the processing unit 104A is split 222 and each portion of the split memories 204A, 204B is associated with different task

information 206A, 206B. For example, the memory 204A may store the task information 206A associated with the clients 108A, 108N while the memory 204B may store the task information 206B associated with clients 108B, 108C. As shown, the task information 206B includes different objects 262A-262N, but these objects are common within the scenes associated with the clients 108B, 108C. Accordingly, systems and methods may be used with embodiments where the processing units are statically split while still providing the benefits of improved computational density.

[0046] Systems and methods of the present disclosure may be used to improve a computational density of one or more processing units, such as GPUs, by grouping users associated with a common underlying background and then scheduling individual processing tasks and/or portions of those tasks during idle times for the processing units. Embodiments may further provide for monitoring to ensure that a minimum level (e.g., a threshold level) of compute capabilities and/or results are provided, and if indicators illustrate that such capability may be lost, may be used to move or otherwise shift clients between different processing units.

[0047] FIG. 3 illustrates an example schematic diagram 300 illustrating embodiments of the present disclosure. While this example is directed toward a rendering task, embodiments of the present disclosure are not limited to only rendering and may be used with a variety of applications in which a known or expected downtime between different components of a task may be estimated. A rendering task A 302 is illustrated to include a series of frames 304, marked as 304A, 304B, and 304N. In this example, the frames 304 may be rendered subsequently to provide content to a user, such as a video sequence or a view of one or more objects within an interaction environment. The rendering task A 302 may be associated with a request submitted by a first client to interact with content. In at least one embodiment, the rendering task A 302 is associated with an environment that includes a set of objects or features that may be rendered, such as certain components, colors, shading, and the like. This set may be saved or accessible by one or more processing units in order to render content on demand responsive to client interactions or instructions.

[0048] Further illustrated is a rendering task B 306 that includes a series of frames 308, marked as 308A, 308B, and 308N. In this example, the frames 308 may also be rendered sequentially such that a first frame is rendered before a second frame, and so forth. As noted herein, one or more features of each of the rendering tasks 302, 306 may be evaluated to estimate or determine a duration of time to perform an action or component of the task, such as to render and provide an individual frame. The rendering task A 302 may have different features than the rendering task B 306 that may change or otherwise influence how long different frames take to render, such as different frame sizes, different resolutions, and the like. Furthermore, one scene may be more complex than another, such as by including additional components. Upon receipt of the rendering task A 302, the task may be assigned to one or more processing units for completion.

[0049] A compute estimation 320 is illustrated including a time 322 corresponding to a horizontal axis. The time 322, and portions thereof, indicate a duration of time to complete one or more components or sub-components of the tasks 302, 306. In this example, the frame 304A may be rendered over a first time 324 (e.g., first duration). The time 324, as

noted herein, may be dependent on features of the frame 304A, such as its size or resolution. Additionally, a frame rate may also be used to estimate a second time 326 (e.g., second duration) between when the frame 304B needs to be rendered in order to provide a level of service expected by the user. That is, the frame rate may determine how many frames per second need to be provided. In an example of 60 fps, a frame is provided approximately every 16.67 ms. In an example of 144 fps, a frame is provided approximately every 6.94 ms. Accordingly, properties of the rendering task A 302 may be used to estimate the second time 326.

[0050] As shown, the first time 324 is less than the second time 326, resulting in an estimated third time 328 (e.g., third duration), which may also be referred to as an idle time. For example, while the duration between rendering the first frame 304A and the second frame 304B may be 16.67 ms, the first time 324 may only take 5 ms. As a result, there is idle time where the processor is not performing any tasks or subtasks and is waiting to render the additional frames in order to provide the requested frame rate. Systems and methods of the present disclosure may use this idle time to render frames for additional users associated with the same underlying background information, thereby reducing the idle time and increasing computational density of the processing unit without allocating additional processing units and/or statically splitting the processing unit.

[0051] In at least one embodiment, the third time 328 may be estimated based, at least, on properties of the rendering task A 302 and then it may be determined whether one or more additional rendering tasks may be performed within the third time 328 such that computational performance associated with the rendering task A 302 is not degraded or affected by the inclusion of additional tasks and/or subtasks. In at least one embodiment, a fourth time 330 (e.g., fourth duration), which may be referred to as a “cushion” or “budget” may be subtracted from the third time 328. For example, some intentional idle time may be desired to account for minor latency or other errors. Accordingly, systems and methods may make a determination as to whether one or more additional tasks may be performed within a fifth time 332 (e.g., fifth duration), corresponding to a difference between the third time 328 and the fourth time 330.

[0052] In the example shown, a frame 308A associated with the rendering task B 306 has a sixth time 334 (e.g., sixth duration) to render the frame. In this example, because the sixth time 334 is less than the fifth time 332 (or than the third time 328 in embodiments where there is no fourth time 330), it may be determined that a portion of the rendering task B 306 may be performed along with the rendering task A 302 without degrading performance of either task 302, 306. Accordingly, systems and methods may then select the frame 308A for rendering between the frames 304A, 304B, thereby adding an additional sub-task within sequential processing of the frames 304A, 304B for the rendering task A 302. In other words, systems and methods may execute one or more processing tasks, such as rendering tasks, where consecutively rendered frames are not associated with the same underlying rendering task. Such a process may then continue as long as there is sufficient time between individual rendering jobs, thereby enabling multiple clients and tasks to use a common processing unit for a common underlying scene.

[0053] FIG. 4A illustrates an example flow chart for an example process 400 to allocate one or more rendering tasks to a GPU, that may be used with embodiments of the present disclosure. It should be understood that for this and other processes presented herein that there can be additional, fewer, or alternative operations performed in similar or alternative order, or at least partially in parallel, within the scope of various embodiments unless otherwise specifically stated. In this example, a first request to render a first sequence of frames representative of a scene is received 402. The request may be a rendering request for an interaction environment in which one or more underlying objects and/or actions may be associated with the interaction environment. A second request to render a second sequence of frames representative of the scene may be received 404. In at least one embodiment, the first request and the second request may be received from different clients. For example, the first request may be from a first client that has selected a first configuration for the scene and the second request may be from a second client that has selected a second configuration for the scene. However, the underlying objects for the scene may have some overlap based on being from a common interaction environment. That is, objects or features loaded into a GPU memory may be common between the scenes associated with the first request and the second request.

[0054] In at least one embodiment, a capacity of a GPU associated with the first request is determined 406. The capacity may refer to an underlying compute capacity with respect to an idle time between competition of a first task and completion of a second task. For example, for rendering, the first task may be rendering a first frame and the second task may be rendering a second frame. It may be determined whether or not there is a threshold available capacity 408. If so, then the second request may be allocated to the GPU 410. As noted herein, allocating the second request to the same GPU executing the first request may enable sharing of resources, such as information stored in GPU memory. Accordingly, the GPU may be used to render a first frame of the first sequence of frames 412. Additionally, the GPU may be used to render a first frame of the second sequence of frames 414 and then to render a second frame of the first sequence of frames 416. In at least one embodiment, the first frame of the second sequence of frames is rendered after the first frame of the first sequence of frames and before a second frame of the first sequence of frames. Furthermore, in various embodiments, the second frame of the first sequence of frames may be a first subsequent frame of the first sequence of frames. If there is insufficient capacity, then the second request may be allocated to a second, different GPU 418.

[0055] FIG. 4B illustrates an example flow chart for an example process 420 to execute portions of different tasks using a common GPU, in accordance with various embodiments. In this example, a first request to perform a first task is received 422. The first task may include a plurality of first sub-tasks. As noted herein, the first task may be associated with a variety of compute tasks that execute processes sequentially at given intervals, including but not limited to rendering tasks, encoding/transcoding tasks, and/or the like. Furthermore, while embodiments may be described with reference to sequential processes, systems and methods of the present disclosure may also be used in processes where tasks are not necessarily sequential or may be performed non-sequentially. A second request is received 424, in which

the second request includes a plurality of second-sub tasks. It may be determined that the first task and the second task share a set of components used in executing the respective tasks **426**. For example, each of the tasks may be associated with an underlying environment that has a common set of features or components. In at least one embodiment, a first duration of time for performing a first individual sub-task of the plurality of first sub-tasks is determined **428** and a second duration of time for performing a second individual sub-task of the plurality of second sub-tasks is also determined **430**. A combined duration of time may be computed, for example by adding the first duration and the second duration, and the combined duration of time may be compared to an idle time **432**. It may be determined that the combined duration of time is less than the idle time. As a result, both the first request and the second request may be assigned to a common processing unit **434** and the processing unit may then perform the first and second requests **436**. For example, in at least one embodiment, the second individual sub-task may be executed after the first individual sub-task but before a subsequent individual sub-task of the first plurality of first sub-tasks. In this manner, the idle time between execution of the first sub-tasks may be occupied with individual tasks of the second sub-tasks to increase computational density of the processing unit.

[0056] FIG. 5A illustrates an example flow chart for an example process **500** to execute multiple processing tasks using a single GPU, in accordance with various embodiments. In this example, it is determined that a processing capacity for a GPU has at least a threshold quantity of idle time between a first operation and a second operation of a first processing task **502**. For example, the idle time may correspond to a duration of time between a first operation to render a first frame and a second operation to render a second, immediately subsequent frame. A second processing task may then be assigned to the GPU for execution during the idle time **504**. For example, a second processing task may also include a number of discrete operations that are performed sequentially, such as rendering frames in a sequence. Thereafter, the GPU may sequentially execute the first operation, at least a portion of the second processing task, and the second operation **506**. For example, the GPU may render a first frame for the first processing task, and then render a first frame for the second processing task before rendering the second frame for the first processing task. In this manner, sequential operations may be separated by different processing tasks when there is sufficient idle time for the GPU between the sequential operations.

[0057] FIG. 5B illustrates an example flow chart for an example process **520** to allocate processing tasks between different processing units, in accordance with various embodiments. In this example, a first processing task is received **522**. At least a portion of the first processing task may be executed using a first processing unit, such as a first operation and a second operation **524**. A first duration of time between a first completion of the first operation and a first start of the second operation may be computed **526**. A second request may be received to execute a second processing task **528**, which may be executed using a second processing unit **530**. For example, a third operation for the second processing task and a fourth operation for the second processing task may be executed on the second processing unit. A second duration of time may be computed between a second completion of the fourth operation and a second

start of the fourth operation, and moreover, it may be determined that the second duration is less than the first duration **532**. As a result, the second task may be allocated for the first processing unit **534**. In at least one embodiment, at least a portion of the second task may be executed after completion of the first operation but before starting the second operation.

[0058] As discussed, aspects of various approaches presented herein can be lightweight enough to execute on a device such as a client device, such as a personal computer or gaming console, in real time. Such processing can be performed on, or for, content that is generated on, or received by, that client device or received from an external source, such as streaming data or other content received over at least one network. In some instances, the processing and/or determination of this content may be performed by one of these other devices, systems, or entities, then provided to the client device (or another such recipient) for presentation or another such use.

[0059] As an example, FIG. 6 illustrates an example network configuration **600** that can be used to provide, generate, modify, encode, process, and/or transmit image data or other such content. In at least one embodiment, a client device **602** can generate or receive data for a session using components of a control application **604** on client device **602** and data stored locally on that client device. In at least one embodiment, a content application **624** executing on a server **620** (e.g., a cloud server or edge server) may initiate a session associated with at least one client device **602**, as may utilize a session manager and user data stored in a user database **636**, and can cause content such as one or more digital assets (e.g., object representations) from an asset repository **634** to be determined by a content manager **626**. A content manager **626** may work with an image synthesis module **628** to generate or synthesize new objects, digital assets, or other such content to be provided for presentation via the client device **602**. In at least one embodiment, this image synthesis module **628** can use one or more neural networks, or machine learning models, which can be trained or updated using a training module **632** or system that is on, or in communication with, the server **620**. This can include training and/or using a diffusion model **630** to generate content tiles that can be used by an image synthesis module **628**, for example, to apply a non-repeating texture to a region of an environment for which image or video data is to be presented via a client device **602**. At least a portion of the generated content may be transmitted to the client device **602** using an appropriate transmission manager **622** to send by download, streaming, or another such transmission channel. An encoder may be used to encode and/or compress at least some of this data before transmitting to the client device **602**. In at least one embodiment, the client device **602** receiving such content can provide this content to a corresponding control application **604**, which may also or alternatively include a graphical user interface **610**, content manager **612**, and image synthesis or diffusion module **614** for use in providing, synthesizing, modifying, or using content for presentation (or other purposes) on or by the client device **602**. A decoder may also be used to decode data received over the network(s) **640** for presentation via client device **602**, such as image or video content through a display **606** and audio, such as sounds and music, through at least one audio playback device **608**, such as speakers or headphones. In at least one embodiment, at least some of this

content may already be stored on, rendered on, or accessible to client device **602** such that transmission over network **640** is not required for at least that portion of content, such as where that content may have been previously downloaded or stored locally on a hard drive or optical disk. In at least one embodiment, a transmission mechanism such as data streaming can be used to transfer this content from server **620**, or user database **636**, to client device **602**. In at least one embodiment, at least a portion of this content can be obtained, enhanced, and/or streamed from another source, such as a third party service **660** or other client device **650**, that may also include a content application **662** for generating, enhancing, or providing content. In at least one embodiment, portions of this functionality can be performed using multiple computing devices, or multiple processors within one or more computing devices, such as may include a combination of CPUs and GPUs.

[0060] In this example, these client devices can include any appropriate computing devices, as may include a desktop computer, notebook computer, set-top box, streaming device, gaming console, smartphone, tablet computer, VR headset, AR goggles, wearable computer, or a smart television. Each client device can submit a request across at least one wired or wireless network, as may include the Internet, an Ethernet, a local area network (LAN), or a cellular network, among other such options. In this example, these requests can be submitted to an address associated with a cloud provider, who may operate or control one or more electronic resources in a cloud provider environment, such as may include a data center or server farm. In at least one embodiment, the request may be received or processed by at least one edge server, that sits on a network edge and is outside at least one security layer associated with the cloud provider environment. In this way, latency can be reduced by enabling the client devices to interact with servers that are in closer proximity, while also improving security of resources in the cloud provider environment.

[0061] In at least one embodiment, such a system can be used for performing graphical rendering operations. In other embodiments, such a system can be used for other purposes, such as for providing image or video content to test or validate autonomous machine applications, or for performing deep learning operations. In at least one embodiment, such a system can be implemented using an edge device, or may incorporate one or more Virtual Machines (VMs). In at least one embodiment, such a system can be implemented at least partially in a data center or at least partially using cloud computing resources.

Inference and Training Logic

[0062] FIG. 7A illustrates inference and/or training logic **715** used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. 7A and/or 7B.

[0063] In at least one embodiment, inference and/or training logic **715** may include, without limitation, code and/or data storage **701** to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **701** to store graph code or other software to control timing and/or order, in

which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs)). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, code and/or data storage **701** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage **701** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0064] In at least one embodiment, any portion of code and/or data storage **701** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **701** may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, choice of whether code and/or data storage **701** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0065] In at least one embodiment, inference and/or training logic **715** may include, without limitation, a code and/or data storage **705** to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage **705** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **705** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs)). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which the code corresponds. In at least one embodiment, any portion of code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. In at least one embodiment, any portion of code and/or data storage **705** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **705** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, choice of whether code and/or data storage **705** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some

other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0066] In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be separate storage structures. In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be same storage structure. In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be partially same storage structure and partially separate storage structures. In at least one embodiment, any portion of code and/or data storage **701** and code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0067] In at least one embodiment, inference and/or training logic **715** may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") **710**, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage **720** that are functions of input/output and/or weight parameter data stored in code and/or data storage **701** and/or code and/or data storage **705**. In at least one embodiment, activations stored in activation storage **720** are generated according to linear algebraic and or matrix-based mathematics performed by ALU(s) **710** in response to performing instructions or other code, wherein weight values stored in code and/or data storage **705** and/or code and/or data storage **701** are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage **705** or code and/or data storage **701** or another storage on or off-chip.

[0068] In at least one embodiment, ALU(s) **710** are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) **710** may be external to a processor or other hardware logic device or circuit that uses them (e.g., a co-processor). In at least one embodiment, ALU(s) **710** may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage **701**, code and/or data storage **705**, and activation storage **720** may be on same processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage **720** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using

a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

[0069] In at least one embodiment, activation storage **720** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., Flash memory), or other storage. In at least one embodiment, activation storage **720** may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, choice of whether activation storage **720** is internal or external to a processor, for example, or comprised of DRAM, SRAM, Flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. 7A may be used in conjunction with an application-specific integrated circuit ("ASIC"), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. 7A may be used in conjunction with central processing unit ("CPU") hardware, graphics processing unit ("GPU") hardware or other hardware, such as field programmable gate arrays ("FPGAs").

[0070] FIG. 7B illustrates inference and/or training logic **715**, according to at least one or more embodiments. In at least one embodiment, inference and/or training logic **715** may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. 7B may be used in conjunction with an application-specific integrated circuit (ASIC), such as Tensorflow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., "Lake Crest") processor from Intel Corp. In at least one embodiment, inference and/or training logic **715** illustrated in FIG. 7B may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic **715** includes, without limitation, code and/or data storage **701** and code and/or data storage **705**, which may be used to store code (e.g., graph code), weight values and/or other information, including bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. 7B, each of code and/or data storage **701** and code and/or data storage **705** is associated with a dedicated computational resource, such as computational hardware **702** and computational hardware **706**, respectively. In at least one embodiment, each of computational hardware **702** and computational hardware **706** comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage **701** and code and/or data storage **705**, respectively, result of which is stored in activation storage **720**.

[0071] In at least one embodiment, each of code and/or data storage **701** and **705** and corresponding computational

hardware **702** and **706**, respectively, correspond to different layers of a neural network, such that resulting activation from one “storage/computational pair **701/702**” of code and/or data storage **701** and computational hardware **702** is provided as an input to “storage/computational pair **705/706**” of code and/or data storage **705** and computational hardware **706**, in order to mirror conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs **701/702** and **705/706** may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not shown) subsequent to or in parallel with storage computation pairs **701/702** and **705/706** may be included in inference and/or training logic **715**.

Data Center

[0072] FIG. 8 illustrates an example data center **800**, in which at least one embodiment may be used. In at least one embodiment, data center **800** includes a data center infrastructure layer **810**, a framework layer **820**, a software layer **830**, and an application layer **840**.

[0073] In at least one embodiment, as shown in FIG. 8, data center infrastructure layer **810** may include a resource orchestrator **812**, grouped computing resources **814**, and node computing resources (“node C.R.s”) **816(1)-816(N)**, where “N” represents any whole, positive integer. In at least one embodiment, node C.R.s **816(1)-816(N)** may include, but are not limited to, any number of central processing units (“CPUs”) or other processors (including accelerators, field programmable gate arrays (FPGAs), graphics processors, etc.), memory devices (e.g., dynamic read-only memory), storage devices (e.g., solid state or disk drives), network input/output (“NW I/O”) devices, network switches, virtual machines (“VMs”), power modules, and cooling modules, etc. In at least one embodiment, one or more node C.R.s from among node C.R.s **816(1)-816(N)** may be a server having one or more of above-mentioned computing resources.

[0074] In at least one embodiment, grouped computing resources **814** may include separate groupings of node C.R.s housed within one or more racks (not shown), or many racks housed in data centers at various geographical locations (also not shown). Separate groupings of node C.R.s within grouped computing resources **814** may include grouped compute, network, memory or storage resources that may be configured or allocated to support one or more workloads. In at least one embodiment, several node C.R.s including CPUs or processors may be grouped within one or more racks to provide compute resources to support one or more workloads. In at least one embodiment, one or more racks may also include any number of power modules, cooling modules, and network switches, in any combination.

[0075] In at least one embodiment, resource orchestrator **812** may configure or otherwise control one or more node C.R.s **816(1)-816(N)** and/or grouped computing resources **814**. In at least one embodiment, resource orchestrator **812** may include a software design infrastructure (“SDI”) management entity for data center **800**. In at least one embodiment, resource orchestrator **812** may include hardware, software or some combination thereof.

[0076] In at least one embodiment, as shown in FIG. 8, framework layer **820** includes a job scheduler **822**, a configuration manager **824**, a resource manager **826** and a distributed file system **828**. In at least one embodiment,

framework layer **820** may include a framework to support software **832** of software layer **830** and/or one or more application(s) **842** of application layer **840**. In at least one embodiment, software **832** or application(s) **842** may respectively include web-based service software or applications, such as those provided by Amazon Web Services, Google Cloud and Microsoft Azure. In at least one embodiment, framework layer **820** may be, but is not limited to, a type of free and open-source software web application framework such as Apache Spark™ (hereinafter “Spark”) that may use distributed file system **828** for large-scale data processing (e.g., “big data”). In at least one embodiment, job scheduler **822** may include a Spark driver to facilitate scheduling of workloads supported by various layers of data center **800**. In at least one embodiment, configuration manager **824** may be capable of configuring different layers such as software layer **830** and framework layer **820** including Spark and distributed file system **828** for supporting large-scale data processing. In at least one embodiment, resource manager **826** may be capable of managing clustered or grouped computing resources mapped to or allocated for support of distributed file system **828** and job scheduler **822**. In at least one embodiment, clustered or grouped computing resources may include grouped computing resource **814** at data center infrastructure layer **810**. In at least one embodiment, resource manager **826** may coordinate with resource orchestrator **812** to manage these mapped or allocated computing resources.

[0077] In at least one embodiment, software **832** included in software layer **830** may include software used by at least portions of node C.R.s **816(1)-816(N)**, grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. The one or more types of software may include, but are not limited to, Internet web page search software, e-mail virus scan software, database software, and streaming video content software.

[0078] In at least one embodiment, application(s) **842** included in application layer **840** may include one or more types of applications used by at least portions of node C.R.s **816(1)-816(N)**, grouped computing resources **814**, and/or distributed file system **828** of framework layer **820**. One or more types of applications may include, but are not limited to, any number of a genomics application, a cognitive compute, and a machine learning application, including training or inferencing software, machine learning framework software (e.g., PyTorch, TensorFlow, Caffe, etc.) or other machine learning applications used in conjunction with one or more embodiments.

[0079] In at least one embodiment, any of configuration manager **824**, resource manager **826**, and resource orchestrator **812** may implement any number and type of self-modifying actions based on any amount and type of data acquired in any technically feasible fashion. In at least one embodiment, self-modifying actions may relieve a data center operator of data center **800** from making possibly bad configuration decisions and possibly avoiding underused and/or poor performing portions of a data center.

[0080] In at least one embodiment, data center **800** may include tools, services, software or other resources to train one or more machine learning models or predict or infer information using one or more machine learning models according to one or more embodiments described herein. For example, in at least one embodiment, a machine learning model may be trained by calculating weight parameters

according to a neural network architecture using software and computing resources described above with respect to data center 800. In at least one embodiment, trained machine learning models corresponding to one or more neural networks may be used to infer or predict information using resources described above with respect to data center 800 by using weight parameters calculated through one or more training techniques described herein.

[0081] In at least one embodiment, data center may use CPUs, application-specific integrated circuits (ASICs), GPUs, FPGAs, or other hardware to perform training and/or inferencing using above-described resources. Moreover, one or more software and/or hardware resources described above may be configured as a service to allow users to train or performing inferencing of information, such as image recognition, speech recognition, or other artificial intelligence services.

[0082] Inference and/or training logic 715 are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment, inference and/or training logic 715 may be used in system FIG. 8 for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0083] Such components can be used for rendering content.

Computer Systems

[0084] FIG. 9 is a block diagram illustrating an exemplary computer system, which may be a system with interconnected devices and components, a system-on-a-chip (SOC) or some combination thereof 900 formed with a processor that may include execution units to execute an instruction, according to at least one embodiment. In at least one embodiment, computer system 900 may include, without limitation, a component, such as a processor 902 to employ execution units including logic to perform algorithms for process data, in accordance with present disclosure, such as in embodiment described herein. In at least one embodiment, computer system 900 may include processors, such as PENTIUM® Processor family, Xeon™, Itanium®, XScale™ and/or StrongARM™, Intel® Core™, or Intel® Nervana™ microprocessors available from Intel Corporation of Santa Clara, California, although other systems (including PCs having other microprocessors, engineering workstations, set-top boxes and like) may also be used. In at least one embodiment, computer system 900 may execute a version of WINDOWS® operating system available from Microsoft Corporation of Redmond, Wash., although other operating systems (UNIX and Linux for example), embedded software, and/or graphical user interfaces, may also be used.

[0085] Embodiments may be used in other devices such as handheld devices and embedded applications. Some examples of handheld devices include cellular phones, Internet Protocol devices, digital cameras, personal digital assistants (“PDAs”), and handheld PCs. In at least one embodiment, embedded applications may include a microcontroller, a digital signal processor (“DSP”), system on a chip, network computers (“NetPCs”), set-top boxes, network hubs,

wide area network (“WAN”) switches, or any other system that may perform one or more instructions in accordance with at least one embodiment.

[0086] In at least one embodiment, computer system 900 may include, without limitation, processor 902 that may include, without limitation, one or more execution units 908 to perform machine learning model training and/or inferencing according to techniques described herein. In at least one embodiment, computer system 900 is a single processor desktop or server system, but in another embodiment computer system 900 may be a multiprocessor system. In at least one embodiment, processor 902 may include, without limitation, a complex instruction set computing (“CISC”) microprocessor, a reduced instruction set computing (“RISC”) microprocessor, a very long instruction word (“VLIW”) computing microprocessor, a processor implementing a combination of instruction sets, or any other processor device, such as a digital signal processor, for example. In at least one embodiment, processor 902 may be coupled to a processor bus 910 that may transmit data signals between processor 902 and other components in computer system 900.

[0087] In at least one embodiment, processor 902 may include, without limitation, a Level 1 (“L1”) internal cache memory (“cache”) 904. In at least one embodiment, processor 902 may have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory may reside external to processor 902. Other embodiments may also include a combination of both internal and external caches depending on particular implementation and needs. In at least one embodiment, register file 906 may store different types of data in various registers including, without limitation, integer registers, floating point registers, status registers, and instruction pointer register.

[0088] In at least one embodiment, execution unit 908, including, without limitation, logic to perform integer and floating point operations, also resides in processor 902. In at least one embodiment, processor 902 may also include a microcode (“ucode”) read only memory (“ROM”) that stores microcode for certain macro instructions. In at least one embodiment, execution unit 908 may include logic to handle a packed instruction set 909. In at least one embodiment, by including packed instruction set 909 in an instruction set of a general-purpose processor 902, along with associated circuitry to execute instructions, operations used by many multimedia applications may be performed using packed data in a general-purpose processor 902. In one or more embodiments, many multimedia applications may be accelerated and executed more efficiently by using full width of a processor’s data bus for performing operations on packed data, which may eliminate need to transfer smaller units of data across processor’s data bus to perform one or more operations one data element at a time.

[0089] In at least one embodiment, execution unit 908 may also be used in microcontrollers, embedded processors, graphics devices, DSPs, and other types of logic circuits. In at least one embodiment, computer system 900 may include, without limitation, a memory 920. In at least one embodiment, memory 920 may be implemented as a Dynamic Random Access Memory (“DRAM”) device, a Static Random Access Memory (“SRAM”) device, flash memory device, or other memory device. In at least one embodiment,

memory 920 may store instruction(s) 919 and/or data 921 represented by data signals that may be executed by processor 902.

[0090] In at least one embodiment, system logic chip may be coupled to processor bus 910 and memory 920. In at least one embodiment, system logic chip may include, without limitation, a memory controller hub (“MCH”) 916, and processor 902 may communicate with MCH 916 via processor bus 910. In at least one embodiment, MCH 916 may provide a high bandwidth memory path 918 to memory 920 for instruction and data storage and for storage of graphics commands, data and textures. In at least one embodiment, MCH 916 may direct data signals between processor 902, memory 920, and other components in computer system 900 and to bridge data signals between processor bus 910, memory 920, and a system I/O 922. In at least one embodiment, system logic chip may provide a graphics port for coupling to a graphics controller. In at least one embodiment, MCH 916 may be coupled to memory 920 through a high bandwidth memory path 918 and graphics/video card 912 may be coupled to MCH 916 through an Accelerated Graphics Port (“AGP”) interconnect 914.

[0091] In at least one embodiment, computer system 900 may use system I/O 922 that is a proprietary hub interface bus to couple MCH 916 to I/O controller hub (“ICH”) 930. In at least one embodiment, ICH 930 may provide direct connections to some I/O devices via a local I/O bus. In at least one embodiment, local I/O bus may include, without limitation, a high-speed I/O bus for connecting peripherals to memory 920, chipset, and processor 902. Examples may include, without limitation, an audio controller 929, a firmware hub (“flash BIOS”) 928, a wireless transceiver 926, a data storage 924, a legacy I/O controller 923 containing user input and keyboard interfaces 925, a serial expansion port 927, such as Universal Serial Bus (“USB”), and a network controller 934. Data storage 924 may comprise a hard disk drive, a floppy disk drive, a CD-ROM device, a flash memory device, or other mass storage device.

[0092] In at least one embodiment, FIG. 9 illustrates a system, which includes interconnected hardware devices or “chips”, whereas in other embodiments, FIG. 9 may illustrate an exemplary System on a Chip (“SoC”). In at least one embodiment, devices may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of computer system 900 are interconnected using compute express link (CXL) interconnects.

[0093] Inference and/or training logic 715 are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment, inference and/or training logic 715 may be used in system FIG. 9 for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0094] Such components can be used for rendering content.

[0095] FIG. 10 is a block diagram illustrating an electronic device 1000 for utilizing a processor 1010, according to at least one embodiment. In at least one embodiment, elec-

tronic device 1000 may be, for example and without limitation, a notebook, a tower server, a rack server, a blade server, a laptop, a desktop, a tablet, a mobile device, a phone, an embedded computer, or any other suitable electronic device.

[0096] In at least one embodiment, electronic device 1000 may include, without limitation, processor 1010 communicatively coupled to any suitable number or kind of components, peripherals, modules, or devices. In at least one embodiment, processor 1010 coupled using a bus or interface, such as a 1^o C. bus, a System Management Bus (“SMBus”), a Low Pin Count (LPC) bus, a Serial Peripheral Interface (“SPI”), a High Definition Audio (“HDA”) bus, a Serial Advance Technology Attachment (“SATA”) bus, a Universal Serial Bus (“USB”) (versions 1, 2, 3), or a Universal Asynchronous Receiver/Transmitter (“UART”) bus. In at least one embodiment, FIG. 10 illustrates a system, which includes interconnected hardware devices or “chips”, whereas in other embodiments, FIG. 10 may illustrate an exemplary System on a Chip (“SoC”). In at least one embodiment, devices illustrated in FIG. 10 may be interconnected with proprietary interconnects, standardized interconnects (e.g., PCIe) or some combination thereof. In at least one embodiment, one or more components of FIG. 10 are interconnected using compute express link (CXL) interconnects.

[0097] In at least one embodiment, FIG. 10 may include a display 1024, a touch screen 1025, a touch pad 1030, a Near Field Communications unit (“NFC”) 1045, a sensor hub 1040, a thermal sensor 1046, an Express Chipset (“EC”) 1035, a Trusted Platform Module (“TPM”) 1038, BIOS/firmware/flash memory (“BIOS, FW Flash”) 1022, a DSP 1060, a drive 1020 such as a Solid State Disk (“SSD”) or a Hard Disk Drive (“HDD”), a wireless local area network unit (“WLAN”) 1050, a Bluetooth unit 1052, a Wireless Wide Area Network unit (“WWAN”) 1056, a Global Positioning System (GPS) 1055, a camera (“USB 3.0 camera”) 1054 such as a USB 3.0 camera, and/or a Low Power Double Data Rate (“LPDDR”) memory unit (“LPDDR3”) 1015 implemented in, for example, LPDDR3 standard. These components may each be implemented in any suitable manner.

[0098] In at least one embodiment, other components may be communicatively coupled to processor 1010 through components discussed above. In at least one embodiment, an accelerometer 1041, Ambient Light Sensor (“ALS”) 1042, compass 1043, and a gyroscope 1044 may be communicatively coupled to sensor hub 1040. In at least one embodiment, thermal sensor 1039, a fan 1037, a keyboard 1036, and a touch pad 1030 may be communicatively coupled to EC 1035. In at least one embodiment, speakers 1063, headphones 1064, and microphone (“mic”) 1065 may be communicatively coupled to an audio unit (“audio codec and class d amp”) 1062, which may in turn be communicatively coupled to DSP 1060. In at least one embodiment, audio unit 1062 may include, for example and without limitation, an audio coder/decoder (“codec”) and a class D amplifier. In at least one embodiment, SIM card (“SIM”) 1057 may be communicatively coupled to WWAN unit 1056. In at least one embodiment, components such as WLAN unit 1050 and Bluetooth unit 1052, as well as WWAN unit 1056 may be implemented in a Next Generation Form Factor (“NGFF”).

[0099] Inference and/or training logic 715 are used to perform inferencing and/or training operations associated

with one or more embodiments. Details regarding inference and/or training logic 715 are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment, inference and/or training logic 715 may be used in system FIG. 10 for inferencing or predicting operations based, at least in part, on weight parameters calculated using neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0100] Such components can be used for rendering content.

[0101] FIG. 11 is a block diagram of a processing system, according to at least one embodiment. In at least one embodiment, system 1100 includes one or more processor(s) 1102 and one or more graphics processor(s) 1108, and may be a single processor desktop system, a multiprocessor workstation system, or a server system having a large number of processor(s) 1102 or processor core(s) 1107. In at least one embodiment, system 1100 is a processing platform incorporated within a system-on-a-chip (SoC) integrated circuit for use in mobile, handheld, or embedded devices.

[0102] In at least one embodiment, system 1100 can include, or be incorporated within a server-based gaming platform, a game console, including a game and media console, a mobile gaming console, a handheld game console, or an online game console. In at least one embodiment, system 1100 is a mobile phone, smart phone, tablet computing device or mobile Internet device. In at least one embodiment, system 1100 can also include, coupled with, or be integrated within a wearable device, such as a smart watch wearable device, smart eyewear device, augmented reality device, or virtual reality device. In at least one embodiment, system 1100 is a television or set top box device having one or more processor(s) 1102 and a graphical interface generated by one or more graphics processor(s) 1108.

[0103] In at least one embodiment, one or more processor(s) 1102 each include one or more processor core(s) 1107 to process instructions which, when executed, perform operations for system and user software. In at least one embodiment, each of one or more processor core(s) 1107 is configured to process a specific instruction set 1109. In at least one embodiment, instruction set 1109 may facilitate Complex Instruction Set Computing (CISC), Reduced Instruction Set Computing (RISC), or computing via a Very Long Instruction Word (VLIW). In at least one embodiment, processor core(s) 1107 may each process a different instruction set 1109, which may include instructions to facilitate emulation of other instruction sets. In at least one embodiment, processor core(s) 1107 may also include other processing devices, such as a Digital Signal Processor (DSP).

[0104] In at least one embodiment, processor(s) 1102 includes cache memory 1104. In at least one embodiment, processor(s) 1102 can have a single internal cache or multiple levels of internal cache. In at least one embodiment, cache memory is shared among various components of processor(s) 1102. In at least one embodiment, processor(s) 1102 also uses an external cache (e.g., a Level-3 (L3) cache or Last Level Cache (LLC)) (not shown), which may be shared among processor core(s) 1107 using known cache coherency techniques. In at least one embodiment, register file 1106 is additionally included in processor(s) 1102 which may include different types of registers for storing different types of data (e.g., integer registers, floating point registers,

status registers, and an instruction pointer register). In at least one embodiment, register file 1106 may include general-purpose registers or other registers.

[0105] In at least one embodiment, one or more processor(s) 1102 are coupled with one or more interface bus(es) 1110 to transmit communication signals such as address, data, or control signals between processor(s) 1102 and other components in system 1100. In at least one embodiment, interface bus(es) 1110, in one embodiment, can be a processor bus, such as a version of a Direct Media Interface (DMI) bus. In at least one embodiment, interface bus(es) 1110 is not limited to a DMI bus, and may include one or more Peripheral Component Interconnect buses (e.g., PCI, PCI Express), memory busses, or other types of interface busses. In at least one embodiment processor(s) 1102 include an integrated memory controller 1116 and a platform controller hub 1130. In at least one embodiment, memory controller 1116 facilitates communication between a memory device and other components of system 1100, while platform controller hub (PCH) 1130 provides connections to I/O devices via a local I/O bus.

[0106] In at least one embodiment, memory device 1120 can be a dynamic random access memory (DRAM) device, a static random access memory (SRAM) device, flash memory device, phase-change memory device, or some other memory device having suitable performance to serve as process memory. In at least one embodiment memory device 1120 can operate as system memory for system 1100, to store data 1122 and instruction 1121 for use when one or more processor(s) 1102 executes an application or process. In at least one embodiment, memory controller 1116 also couples with an optional external graphics processor 1112, which may communicate with one or more graphics processor(s) 1108 in processor(s) 1102 to perform graphics and media operations. In at least one embodiment, a display device 1111 can connect to processor(s) 1102. In at least one embodiment display device 1111 can include one or more of an internal display device, as in a mobile electronic device or a laptop device or an external display device attached via a display interface (e.g., DisplayPort, etc.). In at least one embodiment, display device 1111 can include a head mounted display (HMD) such as a stereoscopic display device for use in virtual reality (VR) applications or augmented reality (AR) applications.

[0107] In at least one embodiment, platform controller hub 1130 enables peripherals to connect to memory device 1120 and processor(s) 1102 via a high-speed I/O bus. In at least one embodiment, I/O peripherals include, but are not limited to, an audio controller 1146, a network controller 1134, a firmware interface 1128, a wireless transceiver 1126, touch sensors 1125, a data storage device 1124 (e.g., hard disk drive, flash memory, etc.). In at least one embodiment, data storage device 1124 can connect via a storage interface (e.g., SATA) or via a peripheral bus, such as a Peripheral Component Interconnect bus (e.g., PCI, PCI Express). In at least one embodiment, touch sensors 1125 can include touch screen sensors, pressure sensors, or fingerprint sensors. In at least one embodiment, wireless transceiver 1126 can be a Wi-Fi transceiver, a Bluetooth transceiver, or a mobile network transceiver such as a 3G, 4G, or Long Term Evolution (LTE) transceiver. In at least one embodiment, firmware interface 1128 enables communication with system firmware, and can be, for example, a unified extensible firmware interface (UEFI). In at least one embodiment,

network controller **1134** can enable a network connection to a wired network. In at least one embodiment, a high-performance network controller (not shown) couples with interface bus(es) **1110**. In at least one embodiment, audio controller **1146** is a multi-channel high definition audio controller. In at least one embodiment, system **1100** includes an optional legacy I/O controller **1140** for coupling legacy (e.g., Personal System 2 (PS/2)) devices to system. In at least one embodiment, platform controller hub **1130** can also connect to one or more Universal Serial Bus (USB) controller(s) **1142** connect input devices, such as keyboard and mouse **1143** combinations, a camera **1144**, or other USB input devices.

[0108] In at least one embodiment, an instance of memory controller **1116** and platform controller hub **1130** may be integrated into a discreet external graphics processor, such as external graphics processor **1112**. In at least one embodiment, platform controller hub **1130** and/or memory controller **1116** may be external to one or more processor(s) **1102**. For example, in at least one embodiment, system **1100** can include an external memory controller **1116** and platform controller hub **1130**, which may be configured as a memory controller hub and peripheral controller hub within a system chipset that is in communication with processor(s) **1102**.

[0109] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. 7A and/or 7B. In at least one embodiment portions or all of inference and/or training logic **715** may be incorporated into graphics processor **1500**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in a graphics processor. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIGS. 7A and/or 7B. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of a graphics processor to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein.

[0110] Such components can be used for rendering content.

[0111] FIG. 12 is a block diagram of a processor **1200** having one or more processor core(s) **1202A-1202N**, an integrated memory controller **1214**, and an integrated graphics processor **1208**, according to at least one embodiment. In at least one embodiment, processor **1200** can include additional cores up to and including additional core **1202N** represented by dashed lined boxes. In at least one embodiment, each of processor core(s) **1202A-1202N** includes one or more internal cache unit(s) **1204A-1204N**. In at least one embodiment, each processor core also has access to one or more shared cache unit(s) **1206**.

[0112] In at least one embodiment, internal cache unit(s) **1204A-1204N** and shared cache unit(s) **1206** represent a cache memory hierarchy within processor **1200**. In at least one embodiment, cache unit(s) **1204A-1204N** may include at least one level of instruction and data cache within each processor core and one or more levels of shared mid-level cache, such as a Level 2 (L2), Level 3 (L3), Level 4 (L4), or other levels of cache, where a highest level of cache before external memory is classified as an LLC. In at least

one embodiment, cache coherency logic maintains coherency between various cache unit(s) **1206** and **1204A-1204N**.

[0113] In at least one embodiment, processor **1200** may also include a set of one or more bus controller unit(s) **1216** and a system agent core **1210**. In at least one embodiment, one or more bus controller unit(s) **1216** manage a set of peripheral buses, such as one or more PCI or PCI express busses. In at least one embodiment, system agent core **1210** provides management functionality for various processor components. In at least one embodiment, system agent core **1210** includes one or more integrated memory controllers **1214** to manage access to various external memory devices (not shown).

[0114] In at least one embodiment, one or more of processor core(s) **1202A-1202N** include support for simultaneous multi-threading. In at least one embodiment, system agent core **1210** includes components for coordinating and processor core(s) **1202A-1202N** during multi-threaded processing. In at least one embodiment, system agent core **1210** may additionally include a power control unit (PCU), which includes logic and components to regulate one or more power states of processor core(s) **1202A-1202N** and graphics processor **1208**.

[0115] In at least one embodiment, processor **1200** additionally includes graphics processor **1208** to execute graphics processing operations. In at least one embodiment, graphics processor **1208** couples with shared cache unit(s) **1206**, and system agent core **1210**, including one or more integrated memory controllers **1214**. In at least one embodiment, system agent core **1210** also includes a display controller **1211** to drive graphics processor output to one or more coupled displays. In at least one embodiment, display controller **1211** may also be a separate module coupled with graphics processor **1208** via at least one interconnect, or may be integrated within graphics processor **1208**.

[0116] In at least one embodiment, a ring based interconnect unit **1212** is used to couple internal components of processor **1200**. In at least one embodiment, an alternative interconnect unit may be used, such as a point-to-point interconnect, a switched interconnect, or other techniques. In at least one embodiment, graphics processor **1208** couples with a ring based interconnect unit **1212** via an I/O link **1213**.

[0117] In at least one embodiment, I/O link **1213** represents at least one of multiple varieties of I/O interconnects, including an on package I/O interconnect which facilitates communication between various processor components and a high-performance embedded memory module **1218**, such as an eDRAM module. In at least one embodiment, each of processor core(s) **1202A-1202N** and graphics processor **1208** use embedded memory modules **1218** as a shared Last Level Cache.

[0118] In at least one embodiment, processor core(s) **1202A-1202N** are homogenous cores executing a common instruction set architecture. In at least one embodiment, processor core(s) **1202A-1202N** are heterogeneous in terms of instruction set architecture (ISA), where one or more of processor core(s) **1202A-1202N** execute a common instruction set, while one or more other cores of processor core(s) **1202A-1202N** executes a subset of a common instruction set or a different instruction set. In at least one embodiment, processor core(s) **1202A-1202N** are heterogeneous in terms of microarchitecture, where one or more cores having a relatively higher power consumption couple with one or

more power cores having a lower power consumption. In at least one embodiment, processor **1200** can be implemented on one or more chips or as an SoC integrated circuit.

[0119] Inference and/or training logic **715** are used to perform inferencing and/or training operations associated with one or more embodiments. Details regarding inference and/or training logic **715** are provided below in conjunction with FIGS. **7A** and/or **7B**. In at least one embodiment portions or all of inference and/or training logic **715** may be incorporated into processor **1200**. For example, in at least one embodiment, training and/or inferencing techniques described herein may use one or more of ALUs embodied in graphics processor **1208**, graphics core(s) **1202A-1202N**, or other components in FIG. **12**. Moreover, in at least one embodiment, inferencing and/or training operations described herein may be done using logic other than logic illustrated in FIGS. **7A** and/or **7B**. In at least one embodiment, weight parameters may be stored in on-chip or off-chip memory and/or registers (shown or not shown) that configure ALUs of graphics processor **1200** to perform one or more machine learning algorithms, neural network architectures, use cases, or training techniques described herein. **[0120]** Such components can be used for rendering content.

Virtualized Computing Platform

[0121] FIG. **13** is an example data flow diagram for a process **1300** of generating and deploying an image processing and inferencing pipeline, in accordance with at least one embodiment. In at least one embodiment, process **1300** may be deployed for use with imaging devices, processing devices, and/or other device types at one or more facilities **1302**. Process **1300** may be executed within a training system **1304** and/or a deployment system **1306**. In at least one embodiment, training system **1304** may be used to perform training, deployment, and implementation of machine learning models (e.g., neural networks, object detection algorithms, computer vision algorithms, etc.) for use in deployment system **1306**. In at least one embodiment, deployment system **1306** may be configured to offload processing and compute resources among a distributed computing environment to reduce infrastructure requirements at facility **1302**. In at least one embodiment, one or more applications in a pipeline may use or call upon services (e.g., inference, visualization, compute, AI, etc.) of deployment system **1306** during execution of applications.

[0122] In at least one embodiment, some of applications used in advanced processing and inferencing pipelines may use machine learning models or other AI to perform one or more processing steps. In at least one embodiment, machine learning models may be trained at facility **1302** using data **1308** (such as imaging data) generated at facility **1302** (and stored on one or more picture archiving and communication system (PACS) servers at facility **1302**), may be trained using imaging or sequencing data **1308** from another facility (ies), or a combination thereof. In at least one embodiment, training system **1304** may be used to provide applications, services, and/or other resources for generating working, deployable machine learning models for deployment system **1306**.

[0123] In at least one embodiment, model registry **1324** may be backed by object storage that may support versioning and object metadata. In at least one embodiment, object storage may be accessible through, for example, a cloud

storage compatible application programming interface (API) from within a cloud platform. In at least one embodiment, machine learning models within model registry **1324** may be uploaded, listed, modified, or deleted by developers or partners of a system interacting with an API. In at least one embodiment, an API may provide access to methods that allow users with appropriate credentials to associate models with applications, such that models may be executed as part of execution of containerized instantiations of applications.

[0124] In at least one embodiment, training system **1304** (FIG. **13**) may include a scenario where facility **1302** is training their own machine learning model, or has an existing machine learning model that needs to be optimized or updated. In at least one embodiment, imaging data **1308** generated by imaging device(s), sequencing devices, and/or other device types may be received. In at least one embodiment, once imaging data **1308** is received, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for a machine learning model. In at least one embodiment, AI-assisted annotation **1310** may include one or more machine learning models (e.g., convolutional neural networks (CNNs)) that may be trained to generate annotations corresponding to certain types of imaging data **1308** (e.g., from certain devices). In at least one embodiment, AI-assisted annotation **1310** may then be used directly, or may be adjusted or fine-tuned using an annotation tool to generate ground truth data. In at least one embodiment, AI-assisted annotation **1310**, labeled data **1312**, or a combination thereof may be used as ground truth data for training a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model(s) **1316**, and may be used by deployment system **1306**, as described herein.

[0125] In at least one embodiment, a training pipeline may include a scenario where facility **1302** needs a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, an existing machine learning model may be selected from a model registry **1324**. In at least one embodiment, model registry **1324** may include machine learning models trained to perform a variety of different inference tasks on imaging data. In at least one embodiment, machine learning models in model registry **1324** may have been trained on imaging data from different facilities than facility **1302** (e.g., facilities remotely located). In at least one embodiment, machine learning models may have been trained on imaging data from one location, two locations, or any number of locations. In at least one embodiment, when being trained on imaging data from a specific location, training may take place at that location, or at least in a manner that protects confidentiality of imaging data or restricts imaging data from being transferred off-premises. In at least one embodiment, once a model is trained—or partially trained—at one location, a machine learning model may be added to model registry **1324**. In at least one embodiment, a machine learning model may then be retrained, or updated, at any number of other facilities, and a retrained or updated model may be made available in model registry **1324**. In at least one embodiment, a machine learning model may then be selected from model registry

1324—and referred to as output model(s) **1316**—and may be used in deployment system **1306** to perform one or more processing tasks for one or more applications of a deployment system.

[0126] In at least one embodiment, a scenario may include facility **1302** requiring a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **1306**, but facility **1302** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, a machine learning model selected from model registry **1324** may not be fine-tuned or optimized for imaging data **1308** generated at facility **1302** because of differences in populations, robustness of training data used to train a machine learning model, diversity in anomalies of training data, and/or other issues with training data. In at least one embodiment, AI-assisted annotation **1310** may be used to aid in generating annotations corresponding to imaging data **1308** to be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, labeled data **1312** may be used as ground truth data for training a machine learning model. In at least one embodiment, retraining or updating a machine learning model may be referred to as model training **1314**. In at least one embodiment, model training **1314**—e.g., AI-assisted annotation **1310**, labeled data **1312**, or a combination thereof—may be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, a trained machine learning model may be referred to as output model(s) **1316**, and may be used by deployment system **1306**, as described herein.

[0127] In at least one embodiment, deployment system **1306** may include software **1318**, services **1320**, hardware **1322**, and/or other components, features, and functionality. In at least one embodiment, deployment system **1306** may include a software “stack,” such that software **1318** may be built on top of services **1320** and may use services **1320** to perform some or all of processing tasks, and services **1320** and software **1318** may be built on top of hardware **1322** and use hardware **1322** to execute processing, storage, and/or other compute tasks of deployment system **1306**. In at least one embodiment, software **1318** may include any number of different containers, where each container may execute an instantiation of an application. In at least one embodiment, each application may perform one or more processing tasks in an advanced processing and inferencing pipeline (e.g., inferencing, object detection, feature detection, segmentation, image enhancement, calibration, etc.). In at least one embodiment, an advanced processing and inferencing pipeline may be defined based on selections of different containers that are desired or required for processing imaging data **1308**, in addition to containers that receive and configure imaging data for use by each container and/or for use by facility **1302** after processing through a pipeline (e.g., to convert outputs back to a usable data type). In at least one embodiment, a combination of containers within software **1318** (e.g., that make up a pipeline) may be referred to as a virtual instrument (as described in more detail herein), and a virtual instrument may leverage services **1320** and hardware **1322** to execute some or all processing tasks of applications instantiated in containers.

[0128] In at least one embodiment, a data processing pipeline may receive input data (e.g., imaging data **1308**) in

a specific format in response to an inference request (e.g., a request from a user of deployment system **1306**). In at least one embodiment, input data may be representative of one or more images, video, and/or other data representations generated by one or more imaging devices. In at least one embodiment, data may undergo pre-processing as part of data processing pipeline to prepare data for processing by one or more applications. In at least one embodiment, post-processing may be performed on an output of one or more inferencing tasks or other processing tasks of a pipeline to prepare an output data for a next application and/or to prepare output data for transmission and/or use by a user (e.g., as a response to an inference request). In at least one embodiment, inferencing tasks may be performed by one or more machine learning models, such as trained or deployed neural networks, which may include output model(s) **1316** of training system **1304**.

[0129] In at least one embodiment, tasks of data processing pipeline may be encapsulated in a container(s) that each represents a discrete, fully functional instantiation of an application and virtualized computing environment that is able to reference machine learning models. In at least one embodiment, containers or applications may be published into a private (e.g., limited access) area of a container registry (described in more detail herein), and trained or deployed models may be stored in model registry **1324** and associated with one or more applications. In at least one embodiment, images of applications (e.g., container images) may be available in a container registry, and once selected by a user from a container registry for deployment in a pipeline, an image may be used to generate a container for an instantiation of an application for use by a user's system.

[0130] In at least one embodiment, developers (e.g., software developers, clinicians, doctors, etc.) may develop, publish, and store applications (e.g., as containers) for performing image processing and/or inferencing on supplied data. In at least one embodiment, development, publishing, and/or storing may be performed using a software development kit (SDK) associated with a system (e.g., to ensure that an application and/or container developed is compliant with or compatible with a system). In at least one embodiment, an application that is developed may be tested locally (e.g., at a first facility, on data from a first facility) with an SDK which may support at least some of services **1320** as a system (e.g., processor **1200** of FIG. 12). In at least one embodiment, because DICOM objects may contain anywhere from one to hundreds of images or other data types, and due to a variation in data, a developer may be responsible for managing (e.g., setting constructs for, building pre-processing into an application, etc.) extraction and preparation of incoming data. In at least one embodiment, once validated by process **1300** (e.g., for accuracy), an application may be available in a container registry for selection and/or implementation by a user to perform one or more processing tasks with respect to data at a facility (e.g., a second facility) of a user.

[0131] In at least one embodiment, developers may then share applications or containers through a network for access and use by users of a system (e.g., process **1300** of FIG. 13). In at least one embodiment, completed and validated applications or containers may be stored in a container registry and associated machine learning models may be stored in model registry **1324**. In at least one embodiment, a requesting entity—who provides an inference or image

processing request—may browse a container registry and/or model registry **1324** for an application, container, dataset, machine learning model, etc., select a desired combination of elements for inclusion in data processing pipeline, and submit an imaging processing request. In at least one embodiment, a request may include input data (and associated patient data, in some examples) that is necessary to perform a request, and/or may include a selection of application(s) and/or machine learning models to be executed in processing a request. In at least one embodiment, a request may then be passed to one or more components of deployment system **1306** (e.g., a cloud) to perform processing of data processing pipeline. In at least one embodiment, processing by deployment system **1306** may include referencing selected elements (e.g., applications, containers, models, etc.) from a container registry and/or model registry **1324**. In at least one embodiment, once results are generated by a pipeline, results may be returned to a user for reference (e.g., for viewing in a viewing application suite executing on a local, on-premises workstation or terminal).

[0132] In at least one embodiment, to aid in processing or execution of applications or containers in pipelines, services **1320** may be leveraged. In at least one embodiment, services **1320** may include compute services, artificial intelligence (AI) services, visualization services, and/or other service types. In at least one embodiment, services **1320** may provide functionality that is common to one or more applications in software **1318**, so functionality may be abstracted to a service that may be called upon or leveraged by applications. In at least one embodiment, functionality provided by services **1320** may run dynamically and more efficiently, while also scaling well by allowing applications to process data in parallel (e.g., using a parallel computing platform **1230** (FIG. 12)). In at least one embodiment, rather than each application that shares a same functionality offered by services **1320** being required to have a respective instance of services **1320**, services **1320** may be shared between and among various applications. In at least one embodiment, services may include an inference server or engine that may be used for executing detection or segmentation tasks, as non-limiting examples. In at least one embodiment, a model training service may be included that may provide machine learning model training and/or retraining capabilities. In at least one embodiment, a data augmentation service may further be included that may provide GPU accelerated data (e.g., DICOM, RIS, CIS, REST compliant, RPC, raw, etc.) extraction, resizing, scaling, and/or other augmentation. In at least one embodiment, a visualization service may be used that may add image rendering effects—such as ray-tracing, rasterization, denoising, sharpening, etc.—to add realism to two-dimensional (2D) and/or three-dimensional (3D) models. In at least one embodiment, virtual instrument services may be included that provide for beam-forming, segmentation, inferencing, imaging, and/or support for other applications within pipelines of virtual instruments.

[0133] In at least one embodiment, where services **1320** includes an AI service (e.g., an inference service), one or more machine learning models may be executed by calling upon (e.g., as an API call) an inference service (e.g., an inference server) to execute machine learning model(s), or processing thereof, as part of application execution. In at least one embodiment, where another application includes one or more machine learning models for segmentation

tasks, an application may call upon an inference service to execute machine learning models for performing one or more of processing operations associated with segmentation tasks. In at least one embodiment, software **1318** implementing advanced processing and inferencing pipeline that includes segmentation application and anomaly detection application may be streamlined because each application may call upon a same inference service to perform one or more inferencing tasks.

[0134] In at least one embodiment, hardware **1322** may include GPUs, CPUs, graphics cards, an AI/deep learning system (e.g., an AI supercomputer, such as NVIDIA's DGX), a cloud platform, or a combination thereof. In at least one embodiment, different types of hardware **1322** may be used to provide efficient, purpose-built support for software **1318** and services **1320** in deployment system **1306**. In at least one embodiment, use of GPU processing may be implemented for processing locally (e.g., at facility **1302**), within an AI/deep learning system, in a cloud system, and/or in other processing components of deployment system **1306** to improve efficiency, accuracy, and efficacy of image processing and generation. In at least one embodiment, software **1318** and/or services **1320** may be optimized for GPU processing with respect to deep learning, machine learning, and/or high-performance computing, as non-limiting examples. In at least one embodiment, at least some of computing environment of deployment system **1306** and/or training system **1304** may be executed in a datacenter one or more supercomputers or high performance computing systems, with GPU optimized software (e.g., hardware and software combination of NVIDIA's DGX System). In at least one embodiment, hardware **1322** may include any number of GPUs that may be called upon to perform processing of data in parallel, as described herein. In at least one embodiment, cloud platform may further include GPU processing for GPU-optimized execution of deep learning tasks, machine learning tasks, or other computing tasks. In at least one embodiment, cloud platform (e.g., NVIDIA's NGC) may be executed using an AI/deep learning supercomputer(s) and/or GPU-optimized software (e.g., as provided on NVIDIA's DGX Systems) as a hardware abstraction and scaling platform. In at least one embodiment, cloud platform may integrate an application container clustering system or orchestration system (e.g., KUBERNETES) on multiple GPUs to enable seamless scaling and load balancing.

[0135] FIG. 14 is a system diagram for an example system **1400** for generating and deploying an imaging deployment pipeline, in accordance with at least one embodiment. In at least one embodiment, system **1400** may be used to implement process **1300** of FIG. 13 and/or other processes including advanced processing and inferencing pipelines. In at least one embodiment, system **1400** may include training system **1304** and deployment system **1306**. In at least one embodiment, training system **1304** and deployment system **1306** may be implemented using software **1318**, services **1320**, and/or hardware **1322**, as described herein.

[0136] In at least one embodiment, system **1400** (e.g., training system **1304** and/or deployment system **1306**) may be implemented in a cloud computing environment (e.g., using cloud **1426**). In at least one embodiment, system **1400** may be implemented locally with respect to a healthcare services facility, or as a combination of both cloud and local computing resources. In at least one embodiment, access to APIs

in cloud **1426** may be restricted to authorized users through enacted security measures or protocols. In at least one embodiment, a security protocol may include web tokens that may be signed by an authentication (e.g., AuthN, AuthZ, Gluecon, etc.) service and may carry appropriate authorization. In at least one embodiment, APIs of virtual instruments (described herein), or other instantiations of system **1400**, may be restricted to a set of public IPs that have been vetted or authorized for interaction.

[0137] In at least one embodiment, various components of system **1400** may communicate between and among one another using any of a variety of different network types, including but not limited to local area networks (LANs) and/or wide area networks (WANs) via wired and/or wireless communication protocols. In at least one embodiment, communication between facilities and components of system **1400** (e.g., for transmitting inference requests, for receiving results of inference requests, etc.) may be communicated over data bus(es), wireless data protocols (Wi-Fi), wired data protocols (e.g., Ethernet), etc.

[0138] In at least one embodiment, training system **1304** may execute training pipelines **1404**, similar to those described herein with respect to FIG. **13**. In at least one embodiment, where one or more machine learning models are to be used in deployment pipeline(s) **1410** by deployment system **1306**, training pipelines **1404** may be used to train or retrain one or more (e.g. pre-trained) models, and/or implement one or more of pre-trained models **1406** (e.g., without a need for retraining or updating). In at least one embodiment, as a result of training pipelines **1404**, output model(s) **1316** may be generated. In at least one embodiment, training pipelines **1404** may include any number of processing steps, such as but not limited to imaging data (or other input data) conversion or adaption. In at least one embodiment, for different machine learning models used by deployment system **1306**, different training pipelines **1404** may be used. In at least one embodiment, training pipeline **1404** similar to a first example described with respect to FIG. **13** may be used for a first machine learning model, training pipeline **1404** similar to a second example described with respect to FIG. **13** may be used for a second machine learning model, and training pipeline **1404** similar to a third example described with respect to FIG. **13** may be used for a third machine learning model. In at least one embodiment, any combination of tasks within training system **1304** may be used depending on what is required for each respective machine learning model. In at least one embodiment, one or more of machine learning models may already be trained and ready for deployment so machine learning models may not undergo any processing by training system **1304**, and may be implemented by deployment system **1306**.

[0139] In at least one embodiment, output model(s) **1316** and/or pre-trained models **1406** may include any types of machine learning models depending on implementation or embodiment. In at least one embodiment, and without limitation, machine learning models used by system **1400** may include machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, Long/Short Term Memory (LSTM), Hopfield, Boltzmann,

deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), and/or other types of machine learning models.

[0140] In at least one embodiment, training pipelines **1404** may include AI-assisted annotation, as described in more detail herein with respect to at least FIG. **14B**. In at least one embodiment, labeled data **1312** (e.g., traditional annotation) may be generated by any number of techniques. In at least one embodiment, labels or other annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, another type of program suitable for generating annotations or labels for ground truth, and/or may be hand drawn, in some examples. In at least one embodiment, ground truth data may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines location of labels), and/or a combination thereof. In at least one embodiment, for each instance of imaging data **1308** (or other data type used by machine learning models), there may be corresponding ground truth data generated by training system **1304**. In at least one embodiment, AI-assisted annotation may be performed as part of deployment pipeline(s) **1410**; either in addition to, or in lieu of AI-assisted annotation included in training pipelines **1404**. In at least one embodiment, system **1400** may include a multi-layer platform that may include a software layer (e.g., software **1318**) of diagnostic applications (or other application types) that may perform one or more medical imaging and diagnostic functions. In at least one embodiment, system **1400** may be communicatively coupled to (e.g., via encrypted links) PACS server networks of one or more facilities. In at least one embodiment, system **1400** may be configured to access and referenced data from PACS servers to perform operations, such as training machine learning models, deploying machine learning models, image processing, inferencing, and/or other operations.

[0141] In at least one embodiment, a software layer may be implemented as a secure, encrypted, and/or authenticated API through which applications or containers may be invoked (e.g., called) from an external environment(s) (e.g., facility **1302**). In at least one embodiment, applications may then call or execute one or more services **1320** for performing compute, AI, or visualization tasks associated with respective applications, and software **1318** and/or services **1320** may leverage hardware **1322** to perform processing tasks in an effective and efficient manner. In at least one embodiment, communications sent to, or received by, a training system **1304** and a deployment system **1306** may occur using a pair of DICOM adapters **1402A**, **1402B**.

[0142] In at least one embodiment, deployment system **1306** may execute deployment pipeline(s) **1410**. In at least one embodiment, deployment pipeline(s) **1410** may include any number of applications that may be sequentially, non-sequentially, or otherwise applied to imaging data (and/or other data types) generated by imaging devices, sequencing devices, genomics devices, etc.—including AI-assisted annotation, as described above. In at least one embodiment, as described herein, a deployment pipeline(s) **1410** for an individual device may be referred to as a virtual instrument for a device (e.g., a virtual ultrasound instrument, a virtual

CT scan instrument, a virtual sequencing instrument, etc.). In at least one embodiment, for a single device, there may be more than one deployment pipeline(s) **1410** depending on information desired from data generated by a device. In at least one embodiment, where detections of anomalies are desired from an MRI machine, there may be a first deployment pipeline(s) **1410**, and where image enhancement is desired from output of an MRI machine, there may be a second deployment pipeline(s) **1410**.

[0143] In at least one embodiment, an image generation application may include a processing task that includes use of a machine learning model. In at least one embodiment, a user may desire to use their own machine learning model, or to select a machine learning model from model registry **1324**. In at least one embodiment, a user may implement their own machine learning model or select a machine learning model for inclusion in an application for performing a processing task. In at least one embodiment, applications may be selectable and customizable, and by defining constructs of applications, deployment and implementation of applications for a particular user are presented as a more seamless user experience. In at least one embodiment, by leveraging other features of system **1400**—such as services **1320** and hardware **1322**—deployment pipeline(s) **1410** may be even more user friendly, provide for easier integration, and produce more accurate, efficient, and timely results.

[0144] In at least one embodiment, deployment system **1306** may include a user interface (“UI”) **1414** (e.g., a graphical user interface, a web interface, etc.) that may be used to select applications for inclusion in deployment pipeline(s) **1410**, arrange applications, modify or change applications or parameters or constructs thereof, use and interact with deployment pipeline(s) **1410** during set-up and/or deployment, and/or to otherwise interact with deployment system **1306**. In at least one embodiment, although not illustrated with respect to training system **1304**, UI **1414** (or a different user interface) may be used for selecting models for use in deployment system **1306**, for selecting models for training, or retraining, in training system **1304**, and/or for otherwise interacting with training system **1304**.

[0145] In at least one embodiment, pipeline manager **1412** may be used, in addition to an application orchestration system **1428**, to manage interaction between applications or containers of deployment pipeline(s) **1410** and services **1320** and/or hardware **1322**. In at least one embodiment, pipeline manager **1412** may be configured to facilitate interactions from application to application, from application to services **1320**, and/or from application or service to hardware **1322**. In at least one embodiment, although illustrated as included in software **1318**, this is not intended to be limiting, and in some examples pipeline manager **1412** may be included in services **1320**. In at least one embodiment, application orchestration system **1428** (e.g., Kubernetes, DOCKER, etc.) may include a container orchestration system that may group applications into containers as logical units for coordination, management, scaling, and deployment. In at least one embodiment, by associating applications from deployment pipeline(s) **1410** (e.g., a reconstruction application, a segmentation application, etc.) with individual containers, each application may execute in a self-contained environment (e.g., at a kernel level) to increase speed and efficiency.

[0146] In at least one embodiment, each application and/or container (or image thereof) may be individually developed,

modified, and deployed (e.g., a first user or developer may develop, modify, and deploy a first application and a second user or developer may develop, modify, and deploy a second application separate from a first user or developer), which may allow for focus on, and attention to, a task of a single application and/or container(s) without being hindered by tasks of another application(s) or container(s). In at least one embodiment, communication, and cooperation between different containers or applications may be aided by pipeline manager **1412** and application orchestration system **1428**. In at least one embodiment, so long as an expected input and/or output of each container or application is known by a system (e.g., based on constructs of applications or containers), application orchestration system **1428** and/or pipeline manager **1412** may facilitate communication among and between, and sharing of resources among and between, each of applications or containers. In at least one embodiment, because one or more of applications or containers in deployment pipeline(s) **1410** may share same services and resources, application orchestration system **1428** may orchestrate, load balance, and determine sharing of services or resources between and among various applications or containers. In at least one embodiment, a scheduler may be used to track resource requirements of applications or containers, current usage or planned usage of these resources, and resource availability. In at least one embodiment, a scheduler may thus allocate resources to different applications and distribute resources between and among applications in view of requirements and availability of a system. In some examples, a scheduler (and/or other component of application orchestration system **1428**) may determine resource availability and distribution based on constraints imposed on a system (e.g., user constraints), such as quality of service (QOS), urgency of need for data outputs (e.g., to determine whether to execute real-time processing or delayed processing), etc.

[0147] In at least one embodiment, services **1320** leveraged by and shared by applications or containers in deployment system **1306** may include compute service(s) **1416**, AI service(s) **1418**, visualization service(s) **1420**, and/or other service types. In at least one embodiment, applications may call (e.g., execute) one or more of services **1320** to perform processing operations for an application. In at least one embodiment, compute service(s) **1416** may be leveraged by applications to perform super-computing or other high-performance computing (HPC) tasks. In at least one embodiment, compute service(s) **1416** may be leveraged to perform parallel processing (e.g., using a parallel computing platform **1430**) for processing data through one or more of applications and/or one or more tasks of a single application, substantially simultaneously. In at least one embodiment, parallel computing platform **1430** (e.g., NVIDIA’s CUDA) may enable general purpose computing on GPUs (GPGPU) (e.g., GPUs/Graphics **1422**). In at least one embodiment, a software layer of parallel computing platform **1430** may provide access to virtual instruction sets and parallel computational elements of GPUs, for execution of compute kernels. In at least one embodiment, parallel computing platform **1430** may include memory and, in some embodiments, a memory may be shared between and among multiple containers, and/or between and among different processing tasks within a single container. In at least one embodiment, inter-process communication (IPC) calls may be generated for multiple containers and/or for multiple

processes within a container to use same data from a shared segment of memory of parallel computing platform **1430** (e.g., where multiple different stages of an application or multiple applications are processing same information). In at least one embodiment, rather than making a copy of data and moving data to different locations in memory (e.g., a read/write operation), same data in same location of a memory may be used for any number of processing tasks (e.g., at a same time, at different times, etc.). In at least one embodiment, as data is used to generate new data as a result of processing, this information of a new location of data may be stored and shared between various applications. In at least one embodiment, location of data and a location of updated or modified data may be part of a definition of how a payload is understood within containers.

[0148] In at least one embodiment, AI service(s) **1418** may be leveraged to perform inferencing services for executing machine learning model(s) associated with applications (e.g., tasked with performing one or more processing tasks of an application). In at least one embodiment, AI service(s) **1418** may leverage AI system **1424** to execute machine learning model(s) (e.g., neural networks, such as CNNs) for segmentation, reconstruction, object detection, feature detection, classification, and/or other inferencing tasks. In at least one embodiment, applications of deployment pipeline (s) **1410** may use one or more of output model(s) **1316** from training system **1304** and/or other models of applications to perform inference on imaging data. In at least one embodiment, two or more examples of inferencing using application orchestration system **1428** (e.g., a scheduler) may be available. In at least one embodiment, a first category may include a high priority/low latency path that may achieve higher service level agreements, such as for performing inference on urgent requests during an emergency, or for a radiologist during diagnosis. In at least one embodiment, a second category may include a standard priority path that may be used for requests that may be non-urgent or where analysis may be performed at a later time. In at least one embodiment, application orchestration system **1428** may distribute resources (e.g., services **1320** and/or hardware **1322**) based on priority paths for different inferencing tasks of AI service(s) **1418**.

[0149] In at least one embodiment, shared storage may be mounted to AI service(s) **1418** within system **1400**. In at least one embodiment, shared storage may operate as a cache (or other storage device type) and may be used to process inference requests from applications. In at least one embodiment, when an inference request is submitted, a request may be received by a set of API instances of deployment system **1306**, and one or more instances may be selected (e.g., for best fit, for load balancing, etc.) to process a request. In at least one embodiment, to process a request, a request may be entered into a database, a machine learning model may be located from model registry **1324** if not already in a cache, a validation step may ensure appropriate machine learning model is loaded into a cache (e.g., shared storage), and/or a copy of a model may be saved to a cache. In at least one embodiment, a scheduler (e.g., of pipeline manager **1412**) may be used to launch an application that is referenced in a request if an application is not already running or if there are not enough instances of an application. In at least one embodiment, if an inference server is not already launched to execute a model, an inference server may be launched. Any number of inference servers may be

launched per model. In at least one embodiment, in a pull model, in which inference servers are clustered, models may be cached whenever load balancing is advantageous. In at least one embodiment, inference servers may be statically loaded in corresponding, distributed servers.

[0150] In at least one embodiment, inferencing may be performed using an inference server that runs in a container. In at least one embodiment, an instance of an inference server may be associated with a model (and optionally a plurality of versions of a model). In at least one embodiment, if an instance of an inference server does not exist when a request to perform inference on a model is received, a new instance may be loaded. In at least one embodiment, when starting an inference server, a model may be passed to an inference server such that a same container may be used to serve different models so long as inference server is running as a different instance.

[0151] In at least one embodiment, during application execution, an inference request for a given application may be received, and a container (e.g., hosting an instance of an inference server) may be loaded (if not already), and a start procedure may be called. In at least one embodiment, pre-processing logic in a container may load, decode, and/or perform any additional pre-processing on incoming data (e.g., using a CPU(s) and/or GPU(s)). In at least one embodiment, once data is prepared for inference, a container may perform inference as necessary on data. In at least one embodiment, this may include a single inference call on one image (e.g., a hand X-ray), or may require inference on hundreds of images (e.g., a chest CT). In at least one embodiment, an application may summarize results before completing, which may include, without limitation, a single confidence score, pixel level-segmentation, voxel-level segmentation, generating a visualization, or generating text to summarize findings. In at least one embodiment, different models or applications may be assigned different priorities. For example, some models may have a real-time (TAT<1 min) priority while others may have lower priority (e.g., TAT<10 min). In at least one embodiment, model execution times may be measured from requesting institution or entity and may include partner network traversal time, as well as execution on an inference service.

[0152] In at least one embodiment, transfer of requests between services **1320** and inference applications may be hidden behind a software development kit (SDK), and robust transport may be provide through a queue. In at least one embodiment, a request will be placed in a queue via an API for an individual application/tenant ID combination and an SDK will pull a request from a queue and give a request to an application. In at least one embodiment, a name of a queue may be provided in an environment from where an SDK will pick it up. In at least one embodiment, asynchronous communication through a queue may be useful as it may allow any instance of an application to pick up work as it becomes available. Results may be transferred back through a queue, to ensure no data is lost. In at least one embodiment, queues may also provide an ability to segment work, as highest priority work may go to a queue with most instances of an application connected to it, while lowest priority work may go to a queue with a single instance connected to it that processes tasks in an order received. In at least one embodiment, an application may run on a GPU-accelerated instance generated in cloud **1426**, and an inference service may perform inferencing on a GPU.

[0153] In at least one embodiment, visualization service(s) 1420 may be leveraged to generate visualizations for viewing outputs of applications and/or deployment pipeline(s) 1410. In at least one embodiment, GPUs/Graphics 1422 may be leveraged by visualization service(s) 1420 to generate visualizations. In at least one embodiment, rendering effects, such as ray-tracing, may be implemented by visualization service(s) 1420 to generate higher quality visualizations. In at least one embodiment, visualizations may include, without limitation, 2D image renderings, 3D volume renderings, 3D volume reconstruction, 2D tomographic slices, virtual reality displays, augmented reality displays, etc. In at least one embodiment, virtualized environments may be used to generate a virtual interactive display or environment (e.g., a virtual environment) for interaction by users of a system (e.g., doctors, nurses, radiologists, etc.). In at least one embodiment, visualization service(s) 1420 may include an internal visualizer, cinematics, and/or other rendering or image processing capabilities or functionality (e.g., ray tracing, rasterization, internal optics, etc.).

[0154] In at least one embodiment, hardware 1322 may include GPUs/Graphics 1422, AI system 1424, cloud 1426, and/or any other hardware used for executing training system 1304 and/or deployment system 1306. In at least one embodiment, GPUs/Graphics 1422 (e.g., NVIDIA's TESLA and/or QUADRO GPUs) may include any number of GPUs that may be used for executing processing tasks of compute service(s) 1416, AI service(s) 1418, visualization service(s) 1420, other services, and/or any of features or functionality of software 1318. For example, with respect to AI service(s) 1418, GPUs/Graphics 1422 may be used to perform pre-processing on imaging data (or other data types used by machine learning models), post-processing on outputs of machine learning models, and/or to perform inferencing (e.g., to execute machine learning models). In at least one embodiment, cloud 1426, AI system 1424, and/or other components of system 1400 may use GPUs/Graphics 1422. In at least one embodiment, cloud 1426 may include a GPU-optimized platform for deep learning tasks. In at least one embodiment, AI system 1424 may use GPUs, and cloud 1426—or at least a portion tasked with deep learning or inferencing—may be executed using one or more AI systems 1424. As such, although hardware 1322 is illustrated as discrete components, this is not intended to be limiting, and any components of hardware 1322 may be combined with, or leveraged by, any other components of hardware 1322.

[0155] In at least one embodiment, AI system 1424 may include a purpose-built computing system (e.g., a supercomputer or an HPC) configured for inferencing, deep learning, machine learning, and/or other artificial intelligence tasks. In at least one embodiment, AI system 1424 (e.g., NVIDIA's DGX) may include GPU-optimized software (e.g., a software stack) that may be executed using a plurality of GPUs/Graphics 1422, in addition to CPUs, RAM, storage, and/or other components, features, or functionality. In at least one embodiment, one or more AI systems 1424 may be implemented in cloud 1426 (e.g., in a data center) for performing some or all of AI-based processing tasks of system 1400.

[0156] In at least one embodiment, cloud 1426 may include a GPU-accelerated infrastructure (e.g., NVIDIA's NGC) that may provide a GPU-optimized platform for executing processing tasks of system 1400. In at least one embodiment, cloud 1426 may include an AI system 1424 for

performing one or more of AI-based tasks of system 1400 (e.g., as a hardware abstraction and scaling platform). In at least one embodiment, cloud 1426 may integrate with application orchestration system 1428 leveraging multiple GPUs to enable seamless scaling and load balancing between and among applications and services 1320. In at least one embodiment, cloud 1426 may be tasked with executing at least some of services 1320 of system 1400, including compute service(s) 1416, AI service(s) 1418, and/or visualization service(s) 1420, as described herein. In at least one embodiment, cloud 1426 may perform small and large batch inference (e.g., executing NVIDIA's TENSOR RT), provide an accelerated parallel computing API and platform 1430 (e.g., NVIDIA's CUDA), execute application orchestration system 1428 (e.g., KUBERNETES), provide a graphics rendering API and platform (e.g., for ray-tracing, 2D graphics, 3D graphics, and/or other rendering techniques to produce higher quality cinematics), and/or may provide other functionality for system 1400.

[0157] FIG. 15A illustrates a data flow diagram for a process 1500 to train, retrain, or update a machine learning model, in accordance with at least one embodiment. In at least one embodiment, process 1500 may be executed using, as a non-limiting example, system 1400 of FIG. 14. In at least one embodiment, process 1500 may leverage services and/or hardware as described herein. In at least one embodiment, refined models 1512 generated by process 1500 may be executed by a deployment system for one or more containerized applications in deployment pipelines.

[0158] In at least one embodiment, model training 1514 may include retraining or updating an initial model 1504 (e.g., a pre-trained model) using new training data (e.g., new input data, such as customer dataset 1506, and/or new ground truth data associated with input data). In at least one embodiment, to retrain, or update, initial model 1504, output or loss layer(s) of initial model 1504 may be reset, deleted, and/or replaced with an updated or new output or loss layer(s). In at least one embodiment, initial model 1504 may have previously fine-tuned parameters (e.g., weights and/or biases) that remain from prior training, so training or retraining 1514 may not take as long or require as much processing as training a model from scratch. In at least one embodiment, during model training 1514, by having reset or replaced output or loss layer(s) of initial model 1504, parameters may be updated and re-tuned for a new data set based on loss calculations associated with accuracy of output or loss layer(s) at generating predictions on new, customer dataset 1506.

[0159] In at least one embodiment, pre-trained models 1506 may be stored in a data store, or registry. In at least one embodiment, pre-trained models 1506 may have been trained, at least in part, at one or more facilities other than a facility executing process 1500. In at least one embodiment, to protect privacy and rights of patients, subjects, or clients of different facilities, pre-trained models 1506 may have been trained, on-premise, using customer or patient data generated on-premise. In at least one embodiment, pre-trained models 1306 may be trained using a cloud and/or other hardware, but confidential, privacy protected patient data may not be transferred to, used by, or accessible to any components of a cloud (or other off premise hardware). In at least one embodiment, where pre-trained models 1506 is trained at using patient data from more than one facility, pre-trained models 1506 may have been individually trained

for each facility prior to being trained on patient or customer data from another facility. In at least one embodiment, such as where a customer or patient data has been released of privacy concerns (e.g., by waiver, for experimental use, etc.), or where a customer or patient data is included in a public data set, a customer or patient data from any number of facilities may be used to train pre-trained models **1506** on-premise and/or off premise, such as in a datacenter or other cloud computing infrastructure.

[0160] In at least one embodiment, when selecting applications for use in deployment pipelines, a user may also select machine learning models to be used for specific applications. In at least one embodiment, a user may not have a model for use, so a user may select a pre-trained model to use with an application. In at least one embodiment, pre-trained model may not be optimized for generating accurate results on customer dataset **1506** of a facility of a user (e.g., based on patient diversity, demographics, types of medical imaging devices used, etc.). In at least one embodiment, prior to deploying a pre-trained model into a deployment pipeline for use with an application(s), pre-trained model may be updated, retrained, and/or fine-tuned for use at a respective facility.

[0161] In at least one embodiment, a user may select pre-trained model that is to be updated, retrained, and/or fine-tuned, and this pre-trained model may be referred to as initial model **1504** for a training system within process **1500**. In at least one embodiment, a customer dataset **1506** (e.g., imaging data, genomics data, sequencing data, or other data types generated by devices at a facility) may be used to perform model training (which may include, without limitation, transfer learning) on initial model **1504** to generate refined model **1512**. In at least one embodiment, ground truth data corresponding to customer dataset **1506** may be generated by training system **1304**. In at least one embodiment, ground truth data may be generated, at least in part, by clinicians, scientists, doctors, practitioners, at a facility.

[0162] In at least one embodiment, AI-assisted annotation may be used in some examples to generate ground truth data. In at least one embodiment, AI-assisted annotation (e.g., implemented using an AI-assisted annotation SDK) may leverage machine learning models (e.g., neural networks) to generate suggested or predicted ground truth data for a customer dataset. In at least one embodiment, a user may use annotation tools within a user interface (a graphical user interface (GUI)) on a computing device.

[0163] In at least one embodiment, user **1510** may interact with a GUI via computing device **1508** to edit or fine-tune (auto) annotations. In at least one embodiment, a polygon editing feature may be used to move vertices of a polygon to more accurate or fine-tuned locations.

[0164] In at least one embodiment, once customer dataset **1506** has associated ground truth data, ground truth data (e.g., from AI-assisted annotation, manual labeling, etc.) may be used by during model training to generate refined model **1512**. In at least one embodiment, customer dataset **1506** may be applied to initial model **1504** any number of times, and ground truth data may be used to update parameters of initial model **1504** until an acceptable level of accuracy is attained for refined model **1512**. In at least one embodiment, once refined model **1512** is generated, refined model **1512** may be deployed within one or more deployment pipelines at a facility for performing one or more processing tasks with respect to medical imaging data.

[0165] In at least one embodiment, refined model **1512** may be uploaded to pre-trained models in a model registry to be selected by another facility. In at least one embodiment, this process may be completed at any number of facilities such that refined model **1512** may be further refined on new datasets any number of times to generate a more universal model.

[0166] FIG. **15B** is an example illustration of a client-server architecture **1532** to enhance annotation tools with pre-trained annotation models, in accordance with at least one embodiment. In at least one embodiment, AI-assisted annotation tool **1536** may be instantiated based on a client-server architecture **1532**. In at least one embodiment, AI-assisted annotation tool **1536** in imaging applications may aid radiologists, for example, identify organs and abnormalities. In at least one embodiment, imaging applications may include software tools that help user **1510** to identify, as a non-limiting example, a few extreme points on a particular organ of interest in raw images **1534** (e.g., in a 3D MRI or CT scan) and receive auto-annotated results for all 2D slices of a particular organ. In at least one embodiment, results may be stored in a data store as training data **1538** and used as (for example and without limitation) ground truth data for training. In at least one embodiment, when computing device **1508** sends extreme points for AI-assisted annotation, a deep learning model, for example, may receive this data as input and return inference results of a segmented organ or abnormality. In at least one embodiment, pre-instantiated annotation tools, such as AI-assisted annotation tool **1536** in FIG. **15B**, may be enhanced by making API calls (e.g., API Call **1544**) to a server, such as an Annotation Assistant Server **1540** that may include a set of pre-trained models **1542** stored in an annotation model registry, for example. In at least one embodiment, an annotation model registry may store pre-trained models **1542** (e.g., machine learning models, such as deep learning models) that are pre-trained to perform AI-assisted annotation on a particular organ or abnormality. These models may be further updated by using training pipelines. In at least one embodiment, pre-installed annotation tools may be improved over time as new labeled data is added.

[0167] Various embodiments can be described by the following clauses:

[0168] 1. A computer-implemented method, comprising:

[0169] receiving a first request to graphically render a first sequence of frames representative of a scene;

[0170] receiving a second request to graphically render a second sequence of frames representative of the scene;

[0171] determining that a processing unit assigned to perform the first request has at least a threshold amount of available capacity; and

[0172] causing a second initial frame of the second sequence of frames to be rendered, using the processing unit, after a first initial frame of the first sequence of frames and before a subsequent frame of the first sequence of frames after the first initial frame.

[0173] 2. The computer-implemented method of clause 1, further comprising:

[0174] determining the available capacity is less than the threshold amount; and

[0175] assigning the second request to a second processing unit.

[0176] 3. The computer-implemented method of clause 1, wherein the scene has a common set of background objects shared by the processing unit.

[0177] 4. The computer-implemented method of clause 3, wherein the common set of background objects is stored in a memory accessible during processing of both the first sequence and the second sequence.

[0178] 5. The computer-implemented method of clause 1, further comprising:

[0179] executing the first request for a first period of time; and

[0180] computing a duration of time between a completion of a first sub-task and a start of a second sub-task.

[0181] 6. The computer-implemented method of clause 1, wherein the available capacity is based, at least, on a complexity of the first sequence of frames.

[0182] 7. The computer-implemented method of clause 1, wherein the available capacity is based, at least, on a frame rate for the first sequence of frames.

[0183] 8. The computer-implemented method of clause 1, wherein the threshold amount of available capacity includes a cushion capacity that exceeds a needed capacity to execute the second request.

[0184] 9. A processor, comprising:

[0185] one or more circuits to:

[0186] determine a processing unit has a processing capacity with at least a threshold quantity of idle time between a first operation and a second operation of a first processing task;

[0187] assign a second processing task to the processing unit to be executed during the idle time; and

[0188] cause the processing unit to sequentially execute the first operation, at least a portion of the processing task, and the second operation.

[0189] 10 The processor of clause 9, wherein the threshold quantity of idle time includes a cushion duration and a duration of time between a completion of the first operation and a start of the second operation.

[0190] 11. The processor of clause 9, wherein the first request and the second request are submitted by different clients.

[0191] 12. The processor of clause 9, wherein the first request and the second request share at least a portion of information stored in a commonly accessible memory.

[0192] 13. The processor of clause 9, wherein the one or more circuits are further to:

[0193] receive a request to execute a third processing task;

[0194] determine the processing unit has a second available capacity that is less than the threshold quantity of idle time; and

[0195] assign the third processing task to a second processing unit.

[0196] 14. The processor of clause 9, wherein the processor is comprised in at least one of:

[0197] a system for performing simulation operations;

[0198] a system for performing simulation operations to test or validate autonomous machine applications;

[0199] a system for performing digital twin operations;

[0200] a system for performing light transport simulation;

[0201] a system for rendering graphical output;

[0202] a system for performing deep learning operations;

[0203] a system implemented using an edge device;

[0204] a system for generating or presenting virtual reality (VR) content;

[0205] a system for generating or presenting augmented reality (AR) content;

[0206] a system for generating or presenting mixed reality (MR) content;

[0207] a system incorporating one or more Virtual Machines (VMs);

[0208] a system for performing operations for a conversational AI application;

[0209] a system for performing operations for a generative AI application;

[0210] a system for performing operations using a language model;

[0211] a system for performing one or more generative content operations using a large language model (LLM);

[0212] a system implemented at least partially in a data center;

[0213] a system for performing hardware testing using simulation;

[0214] a system for performing one or more generative content operations using a language model;

[0215] a system for synthetic data generation;

[0216] a collaborative content creation platform for 3D assets; or

[0217] a system implemented at least partially using cloud computing resources.

[0218] 15. A system, comprising:

[0219] one or more processing units to execute a sequential processing task for a second user during an idle time between execution of a first processing task for a first user and a second processing task for the first user, the idle time being determined based, at least, on an expected output result for the first processing task and the second processing task.

[0220] 16. The system of clause 15, wherein the idle time corresponds to a duration of time between a completion of the first processing task and a start of the second processing task.

[0221] 17. The system of clause 16, wherein the idle time further includes a threshold quantity of cushion time exceeding the duration of time.

[0222] 18. The system of clause 15, wherein the sequential processing task, the first processing task, and the second processing task are associated with a common environment.

[0223] 19. The system of clause 18, wherein the common environment includes information for performing the sequential processing task, the first processing task, and the second processing stored in a commonly accessible memory location.

[0224] 20 The system of clause 15, wherein the system is one of:

[0225] a system for performing simulation operations;

[0226] a system for performing simulation operations to test or validate autonomous machine applications;

[0227] a system for performing digital twin operations;

[0228] a system for performing light transport simulation;

[0229] a system for rendering graphical output;

[0230] a system for performing deep learning operations;

[0231] a system implemented using an edge device;

- [0232] a system for generating or presenting virtual reality (VR) content;
 - [0233] a system for generating or presenting augmented reality (AR) content;
 - [0234] a system for generating or presenting mixed reality (MR) content;
 - [0235] a system incorporating one or more Virtual Machines (VMs);
 - [0236] a system for performing operations for a conversational AI application;
 - [0237] a system for performing operations for a generative AI application;
 - [0238] a system for performing operations using a language model;
 - [0239] a system for performing one or more generative content operations using a large language model (LLM);
 - [0240] a system implemented at least partially in a data center;
 - [0241] a system for performing hardware testing using simulation;
 - [0242] a system for performing one or more generative content operations using a language model;
 - [0243] a system for synthetic data generation;
 - [0244] a collaborative content creation platform for 3D assets; or a system implemented at least partially using cloud computing resources.
- [0245] Other variations are within spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.
- [0246] Use of terms “a” and “an” and “the” and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (meaning “including, but not limited to,”) unless otherwise noted. Term “connected,” when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. Use of term “set” (e.g., “a set of items”) or “subset,” unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, term “subset” of a corresponding set does not necessarily denote a proper subset of corresponding set, but subset and corresponding set may be equal.
- [0247] Conjunctive language, such as phrases of form “at least one of A, B, and C,” or “at least one of A, B and C,” unless specifically stated otherwise or otherwise clearly

contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases “at least one of A, B, and C” and “at least one of A, B and C” refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B, and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, term “plurality” indicates a state of being plural (e.g., “a plurality of items” indicates multiple items). A plurality is at least two items, but can be more when so indicated either explicitly or by context. Further, unless stated otherwise or otherwise clear from context, phrase “based on” means “based at least in part on” and not “based solely on.”

[0248] Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being executed) by one or more processors of a computer system, cause computer system to perform operations described herein. A set of non-transitory computer-readable storage media, in at least one embodiment, comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors—for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit (“CPU”) executes some of instructions while a graphics processing unit (“GPU”) executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

[0249] Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes

described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.

[0250] Use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments of disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.

[0251] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

[0252] In description and claims, terms “coupled” and “connected,” along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, “connected” or “coupled” may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. “Coupled” may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0253] Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as “processing,” “computing,” “calculating,” “determining,” or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system’s registers and/or memories into other data similarly represented as physical quantities within computing system’s memories, registers or other such information storage, transmission or display devices.

[0254] In a similar manner, term “processor” may refer to any device or portion of a device that processes electronic data from registers and/or memory and transform that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, “processor” may be a CPU or a GPU. A “computing platform” may comprise one or more processors. As used herein, “software” processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. Terms “system” and “method” are used herein interchangeably insofar as system may embody one or more methods and methods may be considered a system.

[0255] In present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. Obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In some implementations, process of obtaining, acquiring,

receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In another implementation, process of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. References may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, process of providing, outputting, transmitting, sending, or presenting analog or digital data can be accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

[0256] Although discussion above sets forth example implementations of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities are defined above for purposes of discussion, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

[0257] Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

What is claimed is:

1. A computer-implemented method, comprising:
 - receiving a first request to graphically render a first sequence of frames representative of a scene;
 - receiving a second request to graphically render a second sequence of frames representative of the scene;
 - determining that a processing unit assigned to perform the first request has at least a threshold amount of available capacity; and
 - causing a second initial frame of the second sequence of frames to be rendered, using the processing unit, after a first initial frame of the first sequence of frames and before a subsequent frame of the first sequence of frames after the first initial frame.
2. The computer-implemented method of claim 1, further comprising:
 - determining the available capacity is less than the threshold amount; and
 - assigning the second request to a second processing unit.
3. The computer-implemented method of claim 1, wherein the scene has a common set of background objects shared by the processing unit.
4. The computer-implemented method of claim 3, wherein the common set of background objects is stored in a memory accessible during processing of both the first sequence and the second sequence.
5. The computer-implemented method of claim 1, further comprising:
 - executing the first request for a first period of time; and
 - computing a duration of time between a completion of a first sub-task and a start of a second sub-task.
6. The computer-implemented method of claim 1, wherein the available capacity is based, at least, on a complexity of the first sequence of frames.

7. The computer-implemented method of claim 1, wherein the available capacity is based, at least, on a frame rate for the first sequence of frames.

8. The computer-implemented method of claim 1, wherein the threshold amount of available capacity includes a cushion capacity that exceeds a needed capacity to execute the second request.

9. A processor, comprising:

one or more circuits to:

determine a processing unit has a processing capacity with at least a threshold quantity of idle time between a first operation and a second operation of a first processing task;
assign a second processing task to the processing unit to be executed during the idle time; and
cause the processing unit to sequentially execute the first operation, at least a portion of the processing task, and the second operation.

10. The processor of claim 9, wherein the threshold quantity of idle time includes a cushion duration and a duration of time between a completion of the first operation and a start of the second operation.

11. The processor of claim 9, wherein the first request and the second request are submitted by different clients.

12. The processor of claim 9, wherein the first request and the second request share at least a portion of information stored in a commonly accessible memory.

13. The processor of claim 9, wherein the one or more circuits are further to:

receive a request to execute a third processing task;
determine the processing unit has a second available capacity that is less than the threshold quantity of idle time; and
assign the third processing task to a second processing unit.

14. The processor of claim 9, wherein the processor is comprised in at least one of:

a system for performing simulation operations;
a system for performing simulation operations to test or validate autonomous machine applications;
a system for performing digital twin operations;
a system for performing light transport simulation;
a system for rendering graphical output;
a system for performing deep learning operations;
a system implemented using an edge device;
a system for generating or presenting virtual reality (VR) content;
a system for generating or presenting augmented reality (AR) content;
a system for generating or presenting mixed reality (MR) content;
a system incorporating one or more Virtual Machines (VMs);
a system for performing operations for a conversational AI application;
a system for performing operations for a generative AI application;
a system for performing operations using a language model;
a system for performing one or more generative content operations using a large language model (LLM);
a system implemented at least partially in a data center;
a system for performing hardware testing using simulation;

a system for performing one or more generative content operations using a language model;

a system for synthetic data generation;

a collaborative content creation platform for 3D assets; or

a system implemented at least partially using cloud computing resources.

15. A system, comprising:

one or more processing units to execute a sequential processing task for a second user during an idle time between execution of a first processing task for a first user and a second processing task for the first user, the idle time being determined based, at least, on an expected output result for the first processing task and the second processing task.

16. The system of claim 15, wherein the idle time corresponds to a duration of time between a completion of the first processing task and a start of the second processing task.

17. The system of claim 16, wherein the idle time further includes a threshold quantity of cushion time exceeding the duration of time.

18. The system of claim 15, wherein the sequential processing task, the first processing task, and the second processing task are associated with a common environment.

19. The system of claim 18, wherein the common environment includes information for performing the sequential processing task, the first processing task, and the second processing stored in a commonly accessible memory location.

20. The system of claim 15, wherein the system is one of:

a system for performing simulation operations;
a system for performing simulation operations to test or validate autonomous machine applications;
a system for performing digital twin operations;
a system for performing light transport simulation;
a system for rendering graphical output;
a system for performing deep learning operations;
a system implemented using an edge device;
a system for generating or presenting virtual reality (VR) content;
a system for generating or presenting augmented reality (AR) content;
a system for generating or presenting mixed reality (MR) content;
a system incorporating one or more Virtual Machines (VMs);
a system for performing operations for a conversational AI application;
a system for performing operations for a generative AI application;
a system for performing operations using a language model;
a system for performing one or more generative content operations using a large language model (LLM);
a system implemented at least partially in a data center;
a system for performing hardware testing using simulation;
a system for performing one or more generative content operations using a language model;
a system for synthetic data generation;
a collaborative content creation platform for 3D assets; or
a system implemented at least partially using cloud computing resources.